
Test Automation using Selenium

Ahmed Ashraf

Senior Test Automation Engineer @VOIS

- 5 Years of experience
- 4X ISTQB Certified
 - Foundation Level
 - Agile Tester
 - Mobile Applications Tester
 - Test Automation Engineer V2.



[Ahmed Ashraf | LinkedIn](#)



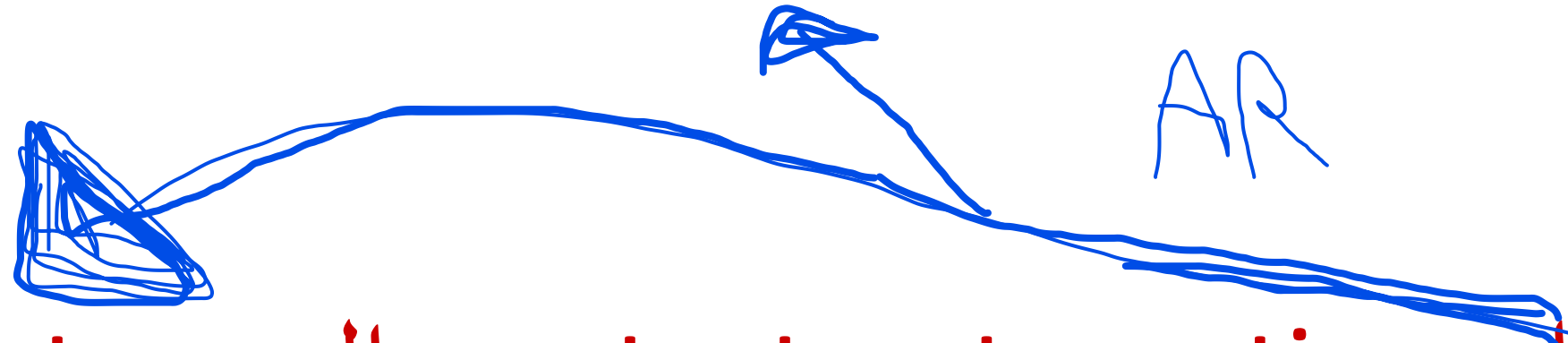
Introduction to Test Automation

What is Testing ?

ER

Pre-conditions > executing steps > comparing > reporting

AR



ولكن هو بيعملي test automation وهو ال execution steps
في ناس فاكروه ان selenium هو ال test automation نقطه واحده من ال

What is Test Automation ?

- Test automation is the practice of **automatically** reviewing and validating a software product by **replacing human effort** with **machine effort**.
- Test automation can also :
 - **enter test data into the System Under Test,**
 - **compare expected and actual results,** and
 - **generate detailed test reports.**



Myths of Test Automation ?

ال test automation مش test level ولا test type خالص

- Test Automation is a Test type or a Test level
برضو هو مش هيعمل replace لل manual مستحيل

• Test Automation will **replace** Manual Testing
بيتشاف برضو ان هو development activity وده غلط تماما لازم يبغي عندك ال mindset بتاعت ال tester

- Test Automation is a **development** activity



Facts of Test Automation ?

- Test Automation can be applied on any **Test Type/Level**
- Test Automation **won't** replace Manual Testing
- Test Automation is a **Testing** activity



Why Test Automation ?

Benefits of Test Automation

ROI → **لینا چاہیے**
Return on investment

Save Time

“Shorter Execution Time”

Repeatability

“More Tests Can Run Per Build”

Reusability

“Cross-Platform Execution”

Increase test
coverage

Save Cost

Improves
accuracy

“less human error”

Early time to
Market

“Quick Feedback”

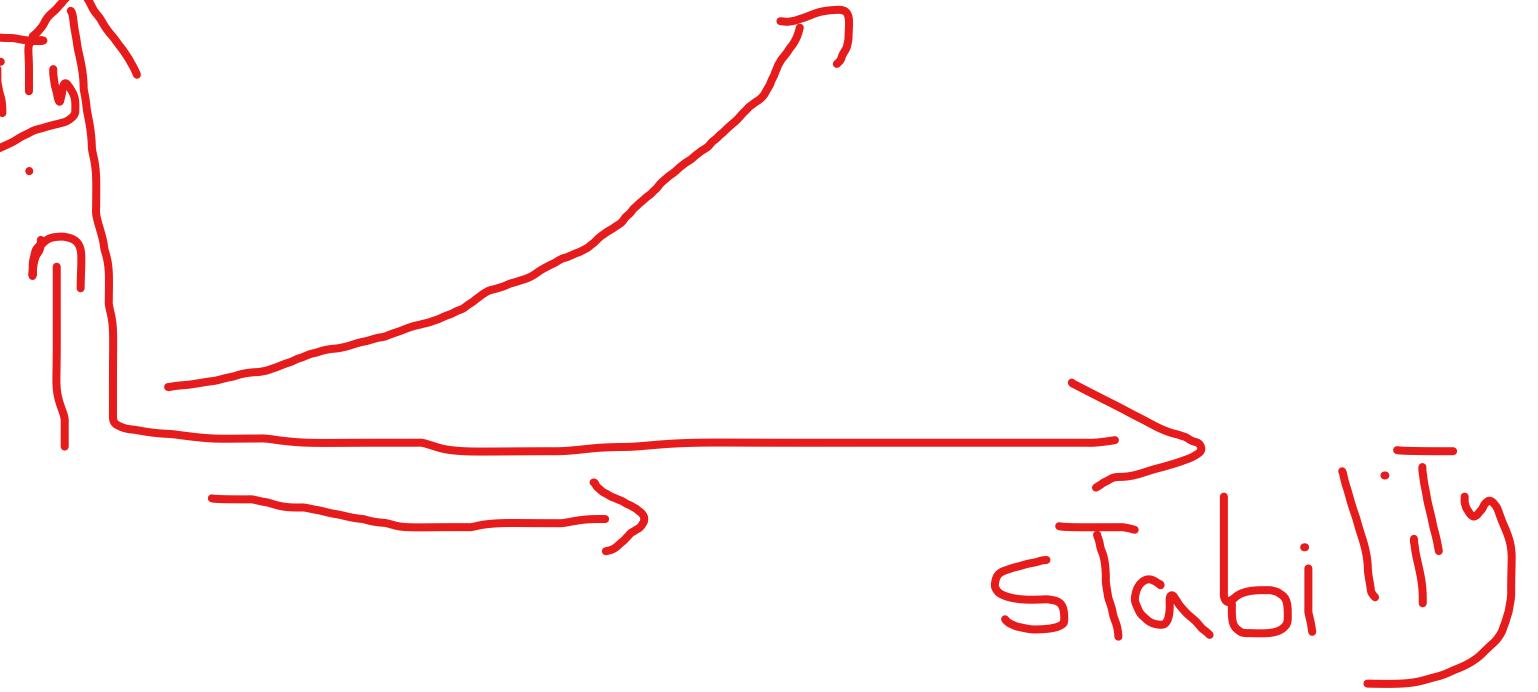
Manual Testing vs Automation Testing

Features	Manual	Automation
Accuracy & Reliability	Low, as manual tests are more prone to human error	High, due to consistent execution of automated scripts
Time (Execution Time)	High	Low
Investment Cost	Low , ROI is low	High , ROI is high
Usage	Suitable for Exploratory, Usability and Ad hoc Testing	Handwritten: فہم و فہم Suitable for Regression Testing
Human Element	Allows for human observation to find out any glitches	No human observation involved Handwritten: ای سی
Handwritten: ۷۷ Customer Experience	Helps in improving the customer experience	Improves speed and coverage but lacks human judgment on user experience

When to do Test Automation ?

When to do Test Automation ?

Productivity



؟ متى ؟

Stability of SUT

SDLC

ROI

Agile
sprints

waterfall

الاجل الى
الوقت

Test Cases suitable to Automate

مع كل change بيحصل بعمل check علي الحته دي انها شغاله زي جزء ال payment

Critical Test
Cases

Difficult to
perform
manually

Repeatedly Test
Cases

Test Cases that
are
time-consuming

Test Cases **not suitable** to Automate

Test Case newly designed

Test Case requirement
changes frequently

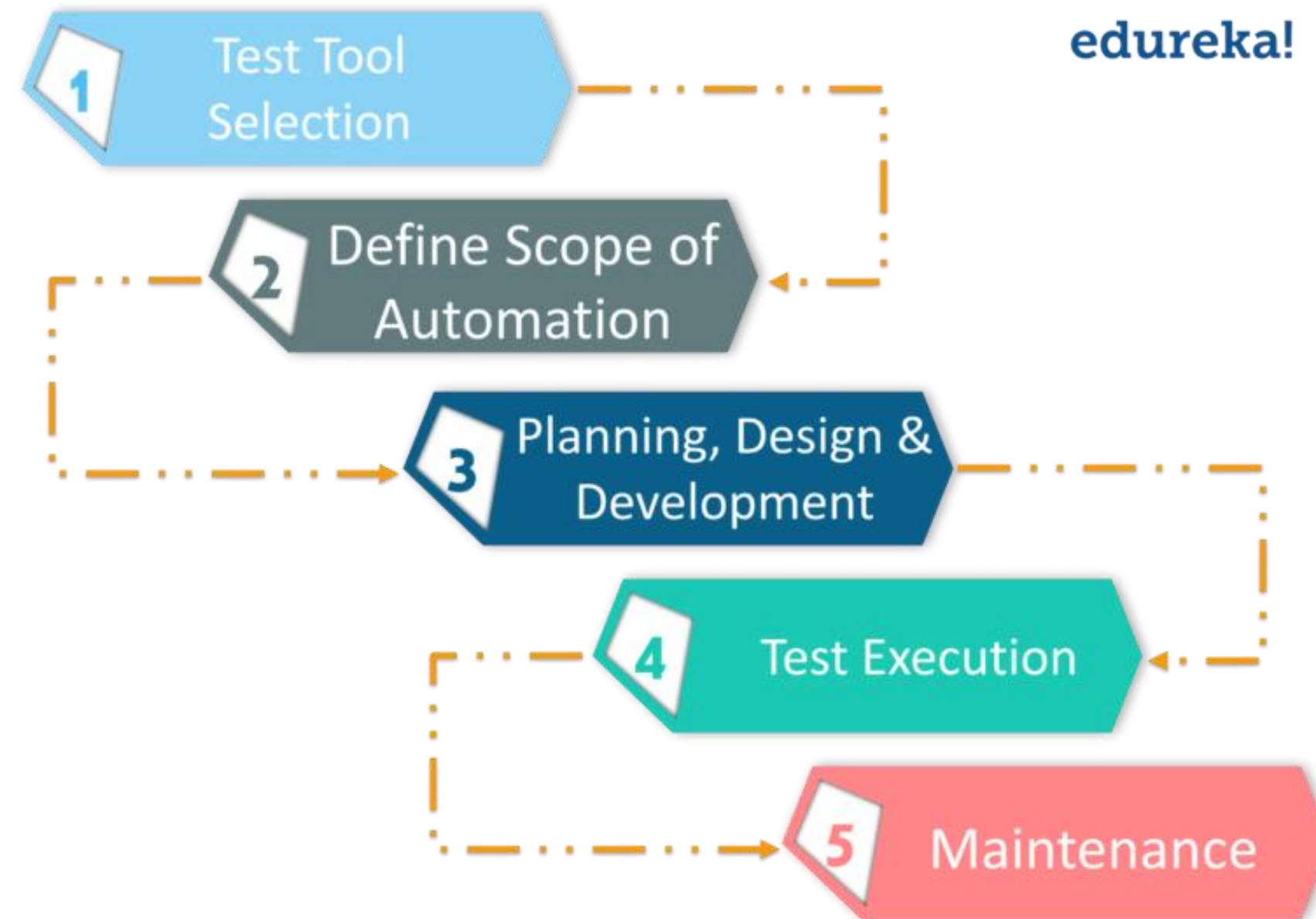
زي ال exploratory حاجه ملهاش requirement و ملهاش test cases

Test cases which are
executed on an ad-hoc
basis

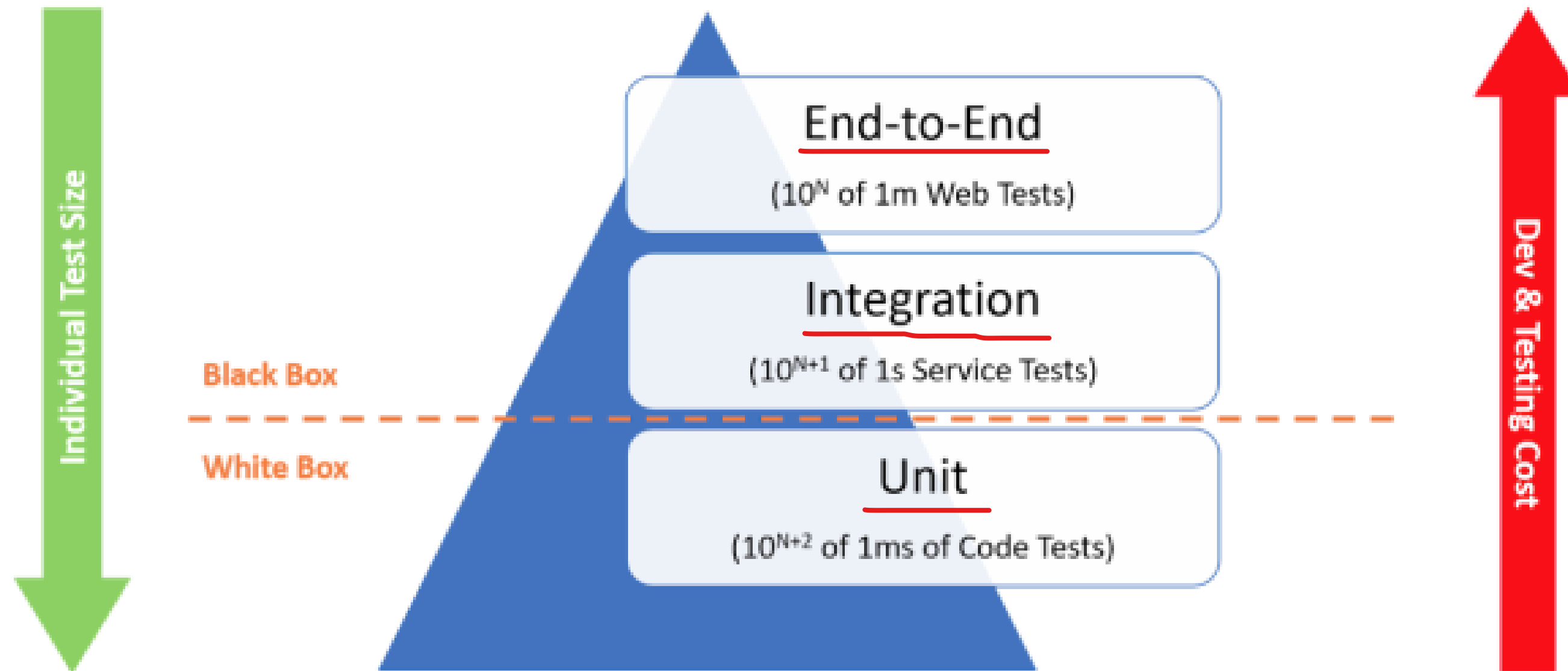
How do you perform Automation Testing?

How do you perform Automation Testing?

2 steps
Automation



How do you perform Automation Testing?



Good Test Automation Practices

Good Practices for Test Automation

- Your Test Automation should be **planned**
- If you started lately works on new features and pull from the oldest ones each sprint
- Keep your eyes always on **user flows** and test cases that will cover **important features**
- Make your automation **fails** when it **actually fail** Don't work to keep it pass
- You can automate all BUT always start from **bottom** as **Test Pyramid diagram**
- Do impact analysis always and refactor your Regression Suite

Selenium



Agenda

- Selenium Introduction
- Selenium Features
- Selenium Components
- How Selenium WebDriver Work

Introduction



What is selenium?

- **Set of tools** support test automation for web applications.
- Dealing with HTML , No backend

ذکر automation مذکرش testing

Selenium automates browsers. That's it!

What you do with that power is entirely up to you.

Primarily it is for **automating web applications** for testing purposes, but is certainly not limited to just that.
Boring web-based administration tasks can (and should) also be automated as well.

Selenium **Features**



Selenium Features

- Open Source
- Support Cross Browser
- Support Several Programming Language: Java, C#, Python and Ruby, ..etc.
- Real Browser Automation
- Extensible, Can be extended with custom functions or integrated into larger automation ecosystems

Selenium Components



Selenium Components

Selenium IDE: Plugin for record/playback.

Selenium Remote Control (RC): Run code in other server.

Selenium WebDriver: Accept commands then send them to browsers.

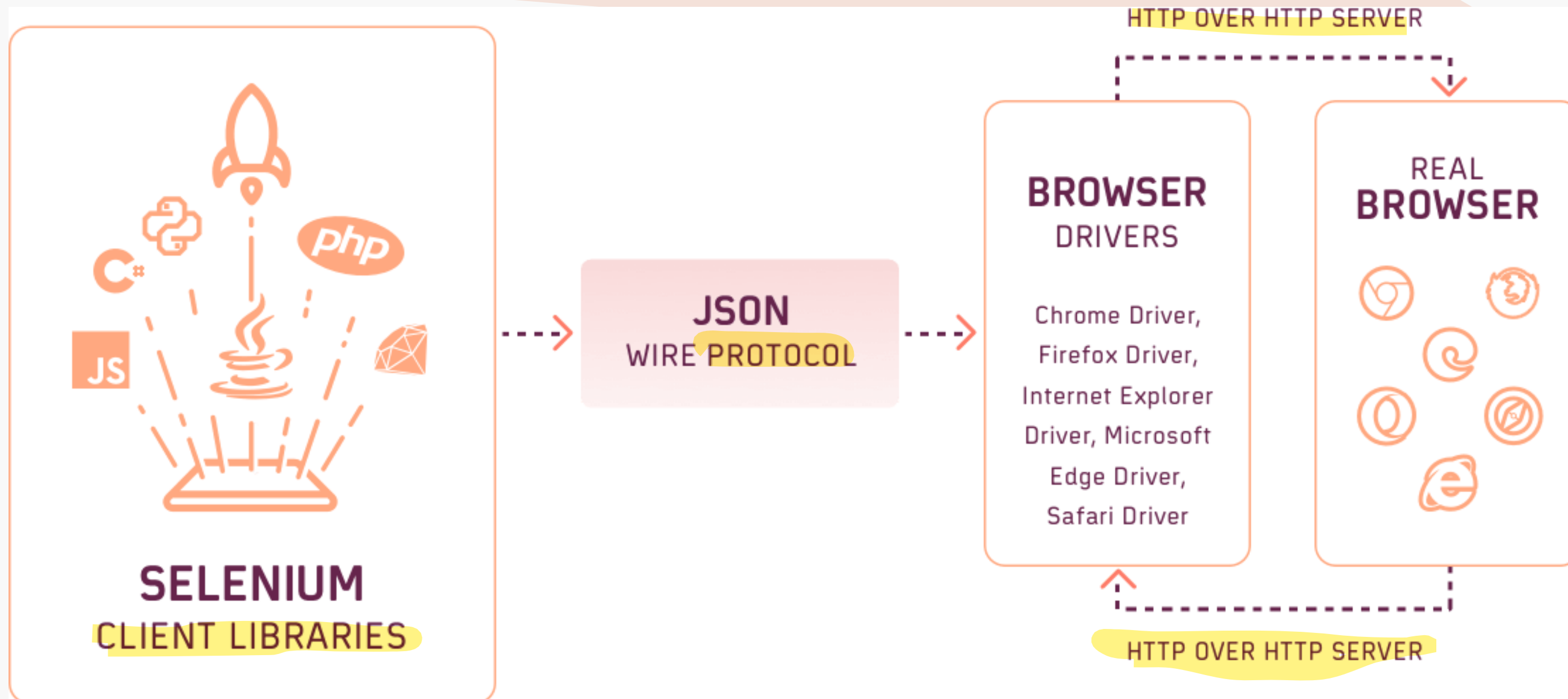
Selenium Grid: Run scripts in multi machines in parallel.

} merged
since
selenium 2

How selenium works ?

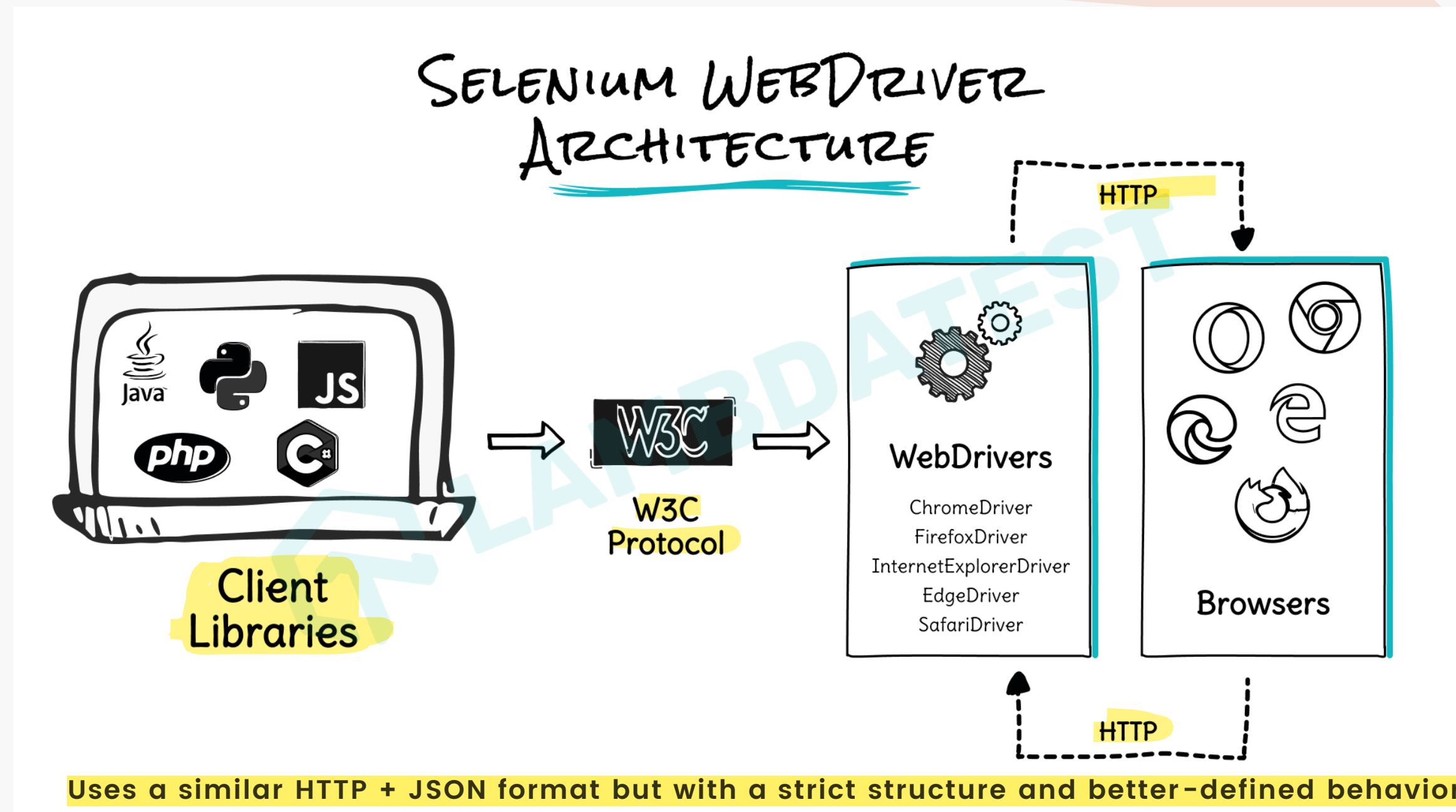


How Selenium WebDriver worked previously (until 3.8)



It converts your test commands (like click, type, navigate) into JSON-formatted HTTP requests.

How Selenium WebDriver Work **now**



- [Selenium WebDriver Architecture Explained - Ultimate Guide On What Is Selenium Architecture And How Does It Work | LambdaTest](#)

Advantages after shifting to Selenium4

- **Flakiness** in web automation can be considerably **reduced**
- The automated Selenium testing will run more consistently across browsers as the browsers, and Selenium WebDriver are using the same protocol.
- The user does not need to customize the test cases based on the browser-specific.
- Action APIs would now let us perform multi-touch actions, zoom-out, zoom-in, pressing two or more keys simultaneously, and many more.

Working with Maven





**What is
Maven?**

What is Maven?

- Maven is a popular build automation and project management tool.
- Maven promotes best practices and conventions, making software development more structured and efficient.





Why

Maven?

Why Maven?

- Simple project setup that follows best practices.
- Managing dependency using pom.xml.
- Automates the build process — compiling code, running tests, generating reports, and packaging (e.g., into a JAR or WAR)
- Maven download the newer versions automatically and store them in local repository.
- Integration with CI/CD Tools
- Plugin Ecosystem



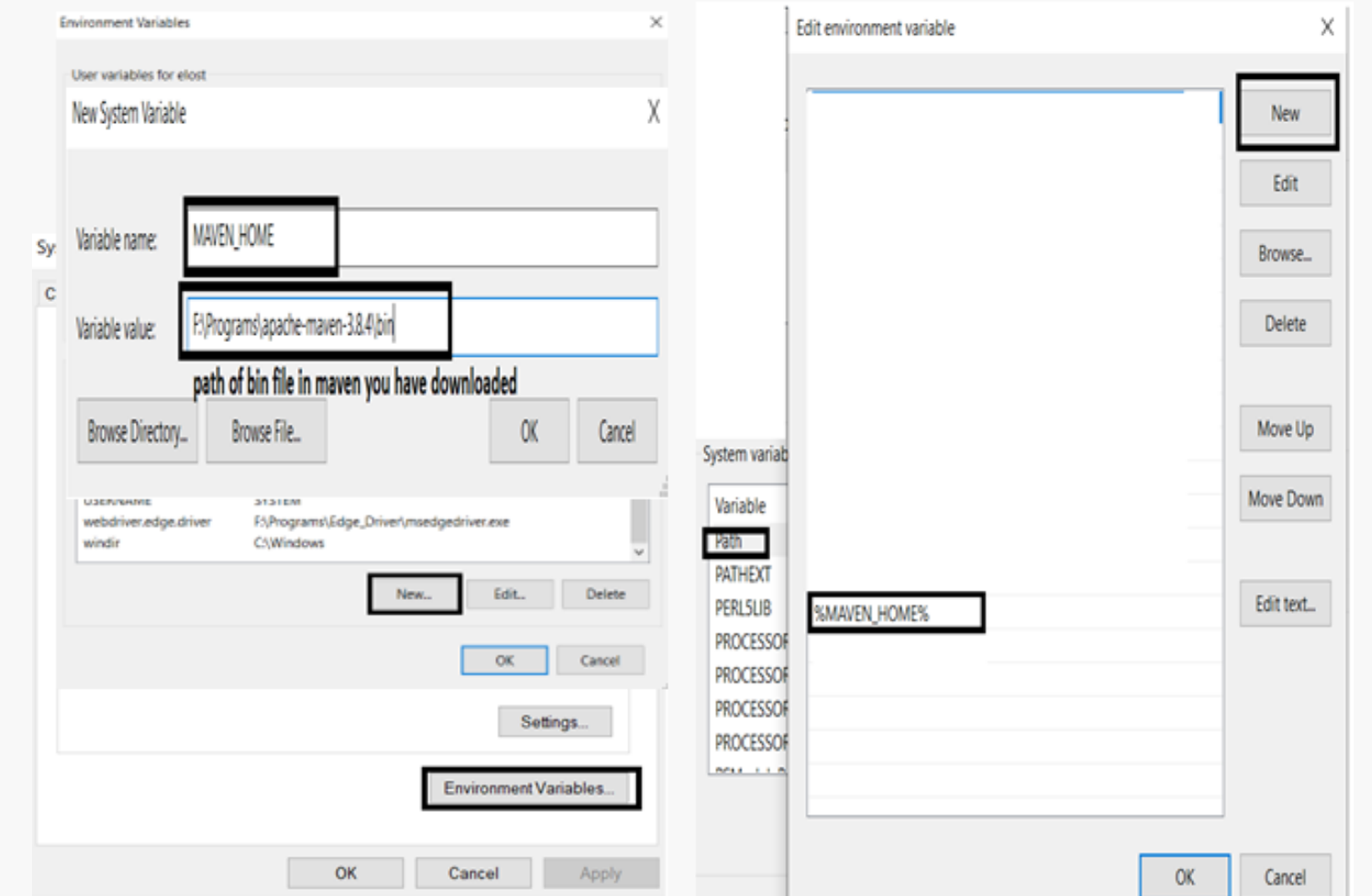
Create Maven Project

Create Maven Project

Download Maven:

- [Maven – Download Apache Maven](#)
- Setup Environment variables

Binary tar.gz archive	apache-maven-3.8.4-bin.tar.gz	apache-maven-3.8.4-bin.tar.gz.sha512	apache-maven-3.8.4-bin.tar.gz.asc
Binary zip archive	apache-maven-3.8.4-bin.zip	apache-maven-3.8.4-bin.zip.sha512	apache-maven-3.8.4-bin.zip.asc
Source tar.gz archive	apache-maven-3.8.4-src.tar.gz	apache-maven-3.8.4-src.tar.gz.sha512	apache-maven-3.8.4-src.tar.gz.asc
Source zip archive	apache-maven-3.8.4-src.zip	apache-maven-3.8.4-src.zip.sha512	apache-maven-3.8.4-src.zip.asc



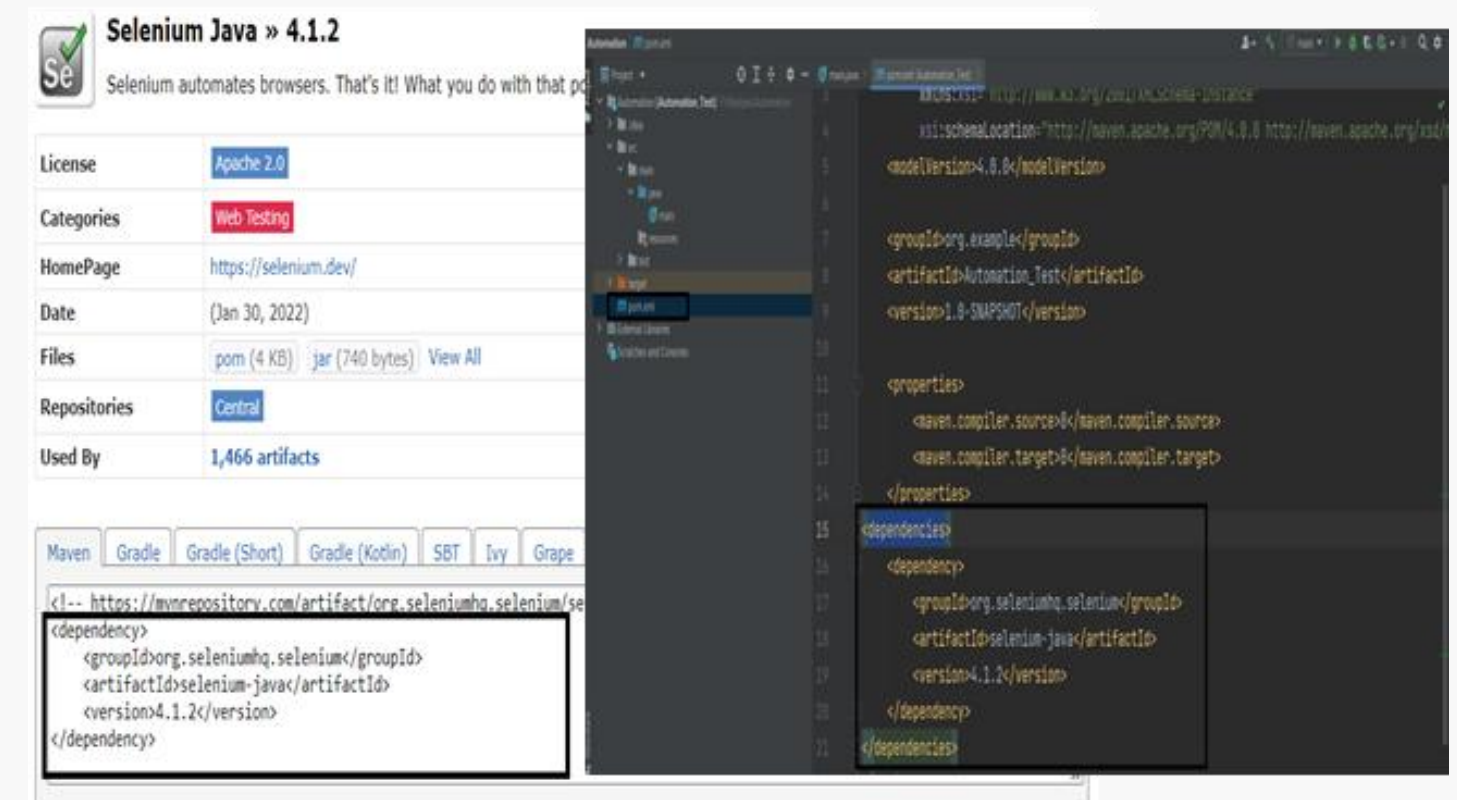
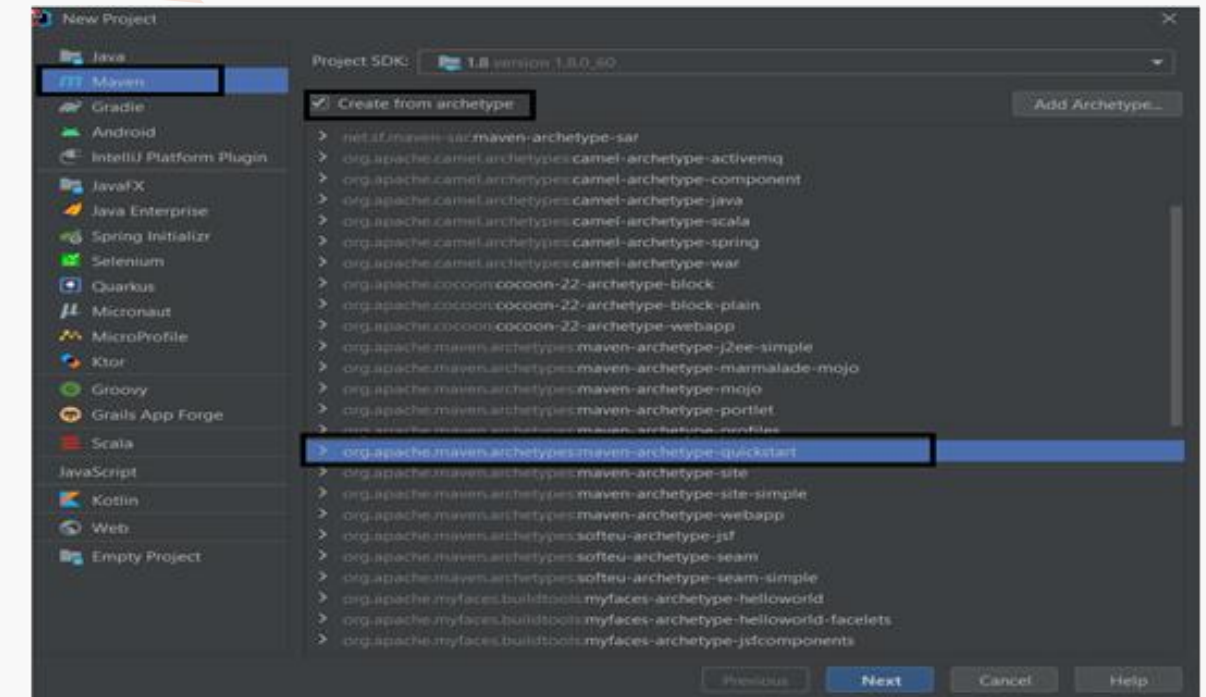
Create Maven Project

Create Maven Project

- [Maven – Download Apache Maven](#)

Add dependencies

- [Maven Repository](#)



Selenium Manager



[Selenium Manager \(Beta\) | Selenium](#)

[WebDriverManager](#)

Selenium Basic Methods



Agenda

- `get("url");`
- `getCurrentUrl();`
- `getTitle();`
- `getPageSource();`
- `quit();`

- `getWindowHandle();`
- `navigate();`
- `manage();`
- `close();`


```
get("url");
```

To launch any Web Application we use Get() to insert the URL as the below code:

```
driver.get("https://www.google.com");
```

browser

Page

navigate() ;

Navigate() method can be used for many operation in browsers

driver.navigate().

to(URL url) = getURL void
to(String url) void
back() void
forward() void
refresh() void
equals(Object obj) boolean
hashCode() int
toString() String
getClass() Class<? extends Navigation>
notify() void
notifyAll() void
wait() void

manage();

The control the screen size of the browser

driver.manage().window().

```
fullscreen() void
getPosition() Point
getSize() → iPhone 4/4 Dimension
maximize() void
minimize() void
setPosition(Point targetPosition) void
setSize(Dimension targetSize) void
hashCode() int
toString() String
getClass() Class<? extends Window>
notify() void
notifyAll() void
wait() void
```



```
getCurrentUrl();
```

To get the URL of the current website launched by

```
driver.getCurrentUrl();
```

```
getTitle();
```

To get the page title of the current website launched

```
driver.getTitle();
```

```
getPageSource();
```

To get the **code or the source** of the **current website**

```
driver.getPageSource();
```



getWindowHandle();

To get the Context unique identifier of the current page .

driver.getWindowHandle();

```
close();
```

The below method will **close the current tab**

```
driver.close();
```



```
quit();
```

The below method will **close All tabs**

```
driver.quit();
```

Web Elements and Locators

Agenda

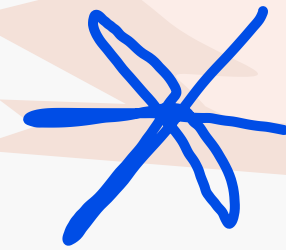


Web Elements



Web Elements Locator (Inspector)



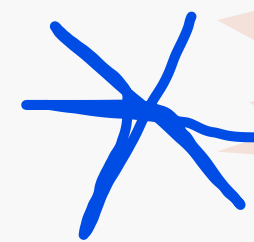


Web Element

- Any individual item existed in web page is called element.
- Every element has its properties.

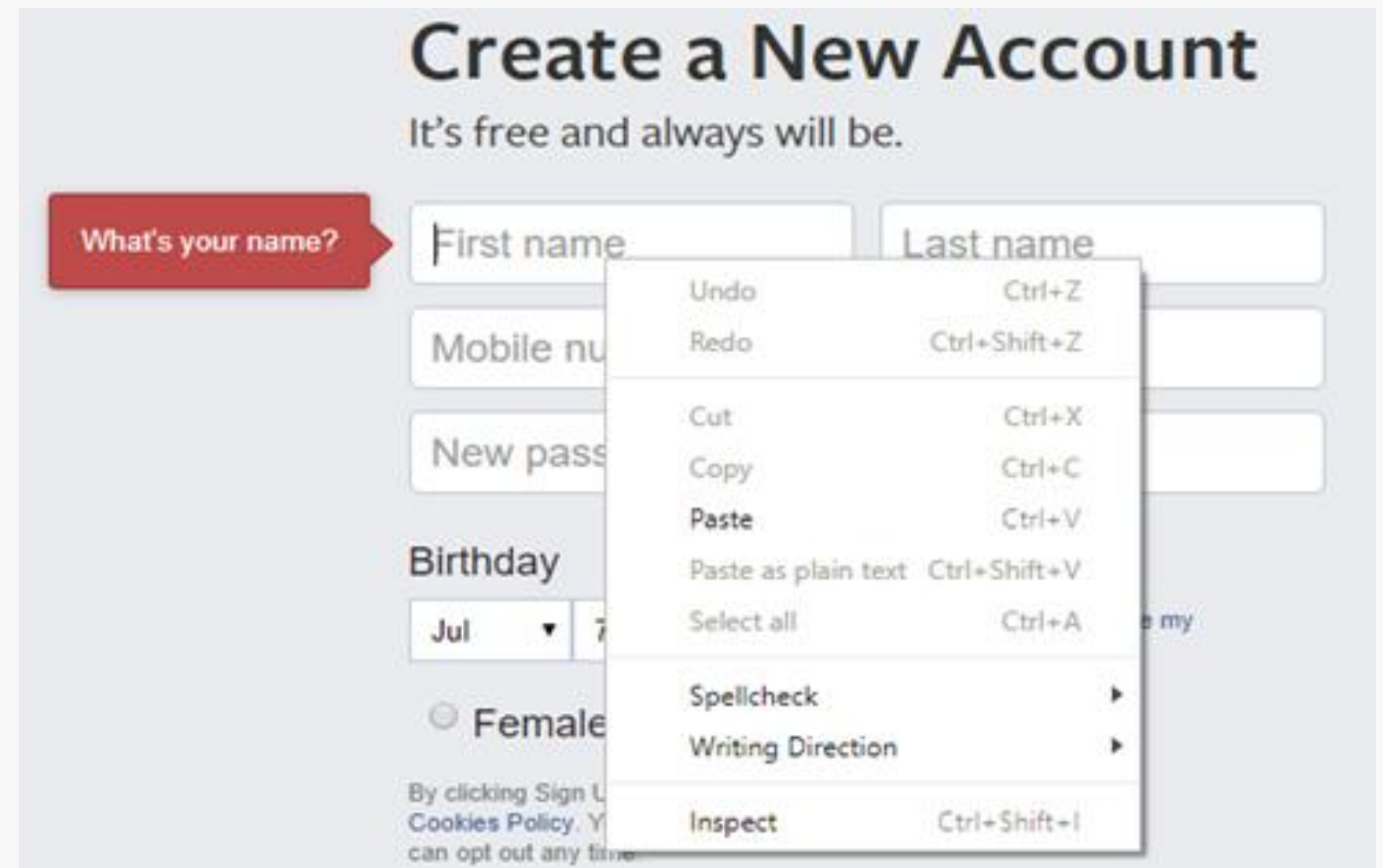
The image displays a 'Sign Up' form and a collection of web UI elements. The form includes fields for 'First name', 'Surname', 'Mobile number or email address', and 'New password'. It also features a 'Date of birth' section with dropdowns for day (14), month (Mar), and year (2022), and a 'Gender' section with radio buttons for 'Female', 'Male', and 'Custom'. A green 'Sign Up' button is at the bottom. Below the form, a collection of UI elements is shown, including two radio buttons labeled 'OPTION 1' and 'OPTION 2', four checkboxes with colored checkmarks, a green 'Button' with a mouse cursor, a text input field with the placeholder 'Your name', and a 'Dropdown' menu with three items: 'Item 1', 'Item 2', and 'Item 3'.

Web Elements Locator (**Inspector**)



Web Elements Locator

- There are many ways to inspect the Elements to know their properties.



Traditional Locators

DOM

- ID
- Name
- Tag Name
- Link Text – Partial link text
- Class Name
- Xpath
- CSS

```
<div>
  <div class="_6lux">
    <input type="text" class="inputtext _55r1 _6luy" name="email" id="email" data-testid="royal_email" placeholder="Email address or phone number" autofocus="1" aria-label="Email address or phone number"> == $0
  </div>
  <div class="_6lux">...</div>
</div>
```

|– ID Locator

ID is one of the important Locator for Selenium to find elements

```
driver.findElement(By.id("Value"));
```




•
-Not all elements have IDs

-The ID might be auto-generated

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password

 Login

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Lighthouse

Recorder

Performance insights

<div id="content" class="large-12 columns">

Styles

Computed

Layout

Filter

element.style {
}
input[type="text"], input[t
input[type="datetime"], in
input[type="month"], input[
input[type="number"], input
input[type="time"], input[t
-webkit-appearance: none;
-webkit-border-radius: 0;
border-radius: 0;
background-color: white;
font-family: inherit;
border: 1px solid #ccc;

DOM

2 - Name Locator

Name is one of the important Locator for Selenium to find elements

```
driver.findElement(By.name("Value"));
```


Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password

➔ Login

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Lighthouse | Recorder | Performance insights

▼<div id="content" class="large-12 columns">

▼<div class="example">

<h2>Login Page</h2>

▶<h4 class="subheader">...</h4>

▼<form name="login" id="login" action="/authenticate" method="post">

▼<div class="row">

::before

▼<div class="large-6 small-12 columns">

<label for="username">Username</label>

.. <input type="text" name="username" id="username"> == \$0

</div>

::after

</div>

▼<div class="row">

Styles | Computed | Layout

Filter

element.style {

}

input[type="text"], input[ty

input[type="datetime"], inpi

input[type="month"], input[

input[type="number"], input[

input[type="time"], input[ti

-webkit-appearance: none;

-webkit-border-radius: 0;

border-radius: 0;

background-color: white

font-family: inherit;

border: 1px solid #ccc



Some elements might have neither ID nor Name In this case, we use other selectors



3 - Class Name

```
<input type="password" class="form-control" autocomplete="off"  
placeholder="Password" name="password" required> == $0
```

```
driver.findElement(By.Class("form-control"))
```

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password





“Compound classes are not allowed in class name locator”

Compound Classes: Classes that have Spaces between the words

نی
value
نی

class "btn -lg btn-primary btn-block"



Class name is not preferred?

because usually more elements may have the same class name

4- Tag Name Locator

```
driver.findElement(By.tagName("button"))
```

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password

➔

 Login

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Lighthouse

Recorder

Performance insights

<h2>Login Page</h2>

▶<h4 class="subheader">...</h4>

▼<form name="login" id="login" action="/authenticate" method="post">

▶<div class="row">...</div>

▶<div class="row">...</div>

▼<button class="radius" type="submit"> == \$0

▼<i class="fa fa-2x fa-sign-in">

::before

" Login"

</i>

</button>

</form>

...

Styles

Computed

L

Filter

element.style {

}

button.radius, .butto

~~-webkit-border-rad:~~

border-radius: ▶ 3px

}

@media only screen an

button, .button {

display: inline-bl

5- Link Text Locator

```
<a href="https://elearning.cis.asu.edu.eg/undergraduate/login/forgot_password.php">Forgotten your username or password?</a> == $0
```

```
driver.findElement(By.linkText("Value"))
```

Link Text Locator

Exit Intent
File Download
File Upload
Floating Menu
Forgot Password
Form Authentication
Frames
Geolocation
Horizontal Slider

```
Elements Console Sources Network Performance Memory A
▶ <li>...</li>
▶ <li>...</li>
▶ <li>...</li>
▶ <li>...</li>
▶ <li>...</li>
▶ <li>...</li>
▼ <li>
. <a href="/login">Form Authentication</a> == $0
  </li>
▶ <li>...</li>
▶ <li>...</li>
▶ <li>...</li>
▶ <li>...</li>
```



Partial Link Text Locator

important

```
<a href="https://elearning.cis.asu.edu.eg/undergraduate/login/forgot_password.php">Forgotten your username or password?</a> == $0
```

```
driver.findElement(By.partiallinktext("Value"))
```

- Partial Link Text Locator

EXAMPLES

- File Download
- File Upload
- Floating Menu
- Forgot Password
- Form Authentication
- Frames
- Geolocation
- Horizontal Slider

```
Elements Console Sources Network Performance Memory A
▶<li>...</li>
▶<li>...</li>
▶<li>...</li>
▶<li>...</li>
▶<li>...</li>
▶<li>...</li>
▼<li>
  . <a href="/login">Form Authentication</a> == $0
  </li>
▶<li>...</li>
▶<li>...</li>
▶<li>...</li>
▶<li>...</li>
```



CSS Locator

CSS using Id

```
driver.findElement(By.cssSelector("#id"));
```

CSS using ClassName

```
driver.findElement(By.cssSelector(".className"));
```

CSS using attribute

```
driver.findElement(By.cssSelector("[attribute=value]"));
```


Combining Attributes

Tag, class, id, and attribute

```
driver.findElement(By.cssSelector("tag.class#id[attribute=value]"));
```

CSS (Sub String)

Starts with: ^

```
driver.findElement(By.cssSelector([attribute^='value']));
```

Ends with: \$

```
driver.findElement(By.cssSelector([attribute$='value']));
```

Contains: *

```
driver.findElement(By.cssSelector([attribute*='value']));
```

CSS Locator

CSS with children “space” (**Direct or Indirect**)

```
driver.findElement(By.cssSelector(".parent .child"));
```

CSS with children “>” (**Direct only**)

```
driver.findElement(By.cssSelector(".parent>.child"));
```

nth-of-type(**index**)

```
driver.findElement(By.cssSelector(".parent .child:nth-of-type(index)"));
```


8-

XPath Locator

XPath is one of the important locators for Selenium to find elements

```
driver.findElement(By.xpath("//tagName[@attribute = 'value']"));
```

```
<input type="search"
```

```
driver.findElement(By.xpath("//input[@type = 'search']"));
```

Note : you can use * to select any tag

```
driver.findElement(By.xpath("//*[@attribute = 'value']"));
```

Absolute XPath vs Relative XPath

Absolute XPath

```
/html/body/div[1]/div[2]/ul/li[3]/a
```

$\Delta \rightarrow$

✖ Cons

- **Fragile – Breaks if any part of the structure changes.**
- **Long and unreadable for deeply nested elements.**

Relative XPath

```
//input[@name='email']
```

XPath Locator

Contains

```
driver.findElement(By.xpath("//*[contains(@attribute,'partial value')]");
```

Text

```
driver.findElement(By.xpath("//*[text()='value']");
```

Or

```
driver.findElement(By.xpath("//*[.='value']");
```

XPath Locator

Child (direct)

```
//input[@name='email'] /input
```

Child (direct or indirect)

```
//input[@name='email'] //input
```

XPath Locator

index

```
//input[@name='email'] /input[3]
```

Handwritten notes in blue ink: "3rd element" with an arrow pointing to the "[3]" in the XPath expression above, and "3rd" written separately to the right.

[How to use XPath in Selenium? \(With Examples\) | BrowserStack](#)

Utilizing Locators

Utilizing Locators

ByChained

```
By example = new ByChained(By.id("login-form"), By.id("username-field"));  
WebElement username_input = driver.findElement(example);
```

ByAll

```
By example = new ByAll(By.id("password-field"), By.id("username-field"));  
List<WebElement> login_inputs = driver.findElements(example);
```

Relative Locators

Very important

Relative Locators

Above

```
By emailLocator = RelativeLocator.with(By.tagName("input")).above(By.id("password"));
```

Below

```
By passwordLocator = RelativeLocator.with(By.tagName("input")).below(By.id("email"));
```

Relative Locators

Left of

```
By cancelLocator = RelativeLocator.with(By.tagName("button")).toLeftOf(By.id("submit"));
```

Right of

```
By submitLocator = RelativeLocator.with(By.tagName("button")).toRightOf(By.id("cancel"));
```

Relative Locators

Near

Max 50px

```
By emailLocator = RelativeLocator.with(By.tagName("input")).near(By.id("lbl-email"));
```

Chaining relative locators

```
By submitLocator = RelativeLocator.with(By.tagName("button")).below(By.id("email")).toRightOf(By.id("cancel"));
```

Tips on working with locators

Tips on working with locators

- **Prefer id when available:**
 - ❑ Fastest and most reliable.
 - ❑ Use when IDs are unique, consistent, and predictable.
- **Fallback to CSS selectors if no reliable ID:**
 - ❑ More readable and easier to debug than XPath.
 - ❑ Perform well across browsers.
- **Use XPath only when necessary:**
 - Very flexible, but:
 - ❑ Syntax is more complex.
 - ❑ Can be harder to maintain and debug.
 - ❑ Typically slower, as it's not performance-optimized by browsers.

Tips on working with locators

- **Use linkText and partialLinkText carefully:**
 - ❑ Only work on anchor (<a>) tags.
 - ❑ Internally use querySelectorAll.
- **Avoid using tagName as a primary locator:**
 - ❑ Tags like <div>, , etc. are often repeated.
 - ❑ Useful with findElements() to get a list of similar elements.
- **Keep locators compact and readable:**
 - ❑ Avoid long DOM traversal chains.
 - ❑ Simpler locators reduce flakiness and improve performance.
- **Scope your locators whenever possible:**
 - ❑ Narrow the context to avoid unnecessary DOM traversal.
 - ❑ Example: Use element.findElement(By...) instead of global search when possible

[Tips on working with locators | Selenium](#)



Web Element vs By

Web Element vs By

By

الطريق
→

- A way to define how to locate elements.
- Used to locate elements, but doesn't represent the actual element.

Web Element

العنصر
→

- Represents the actual element found on the page.
- An object that allows interaction with the real UI element.

Why @FindBy is Flaky

Why @FindBy is Flaky

- Why @FindBy is Flaky (Even with Relocation)

// Timeline of execution:

t0: Test thread calls element.click()

t1: PageFactory proxy initiates findElement()

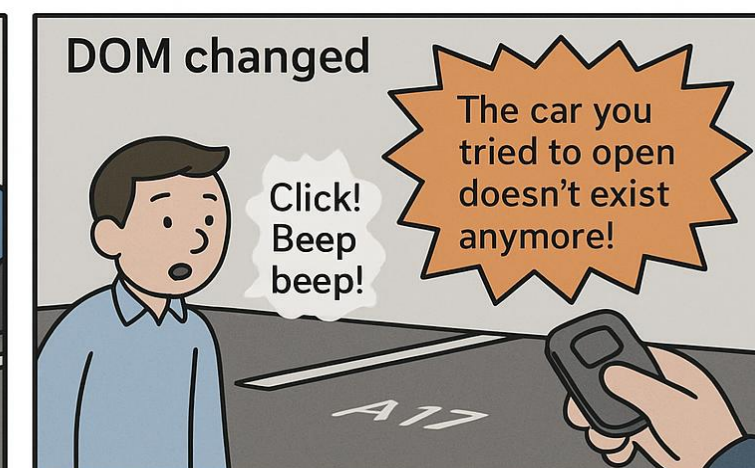
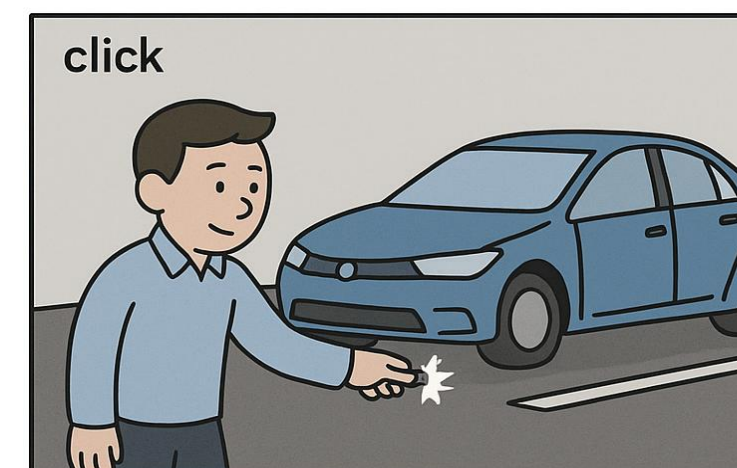
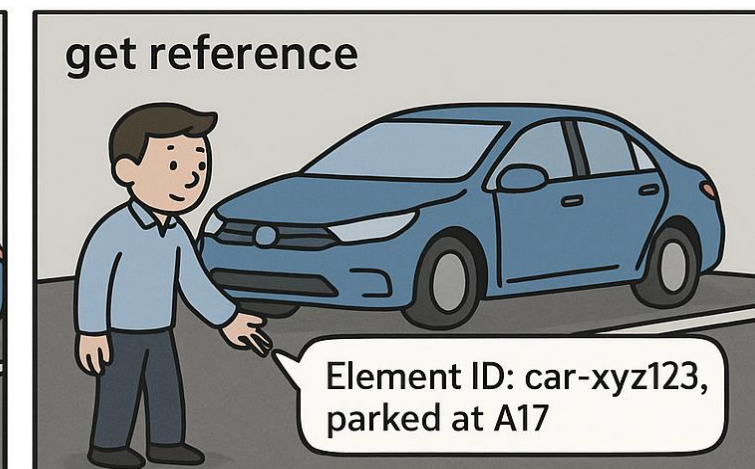
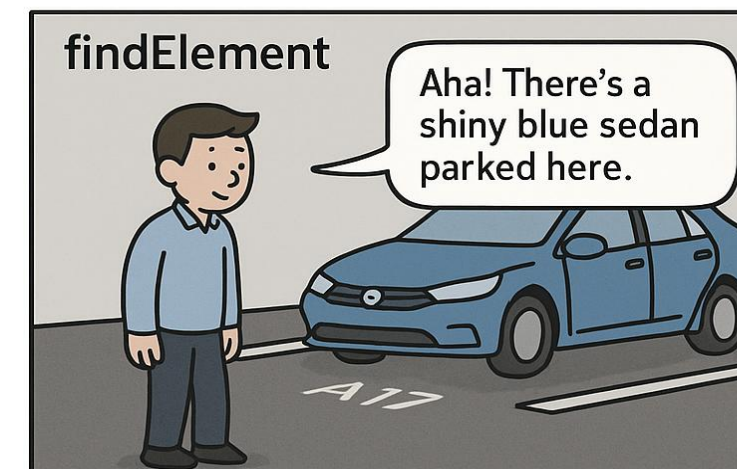
t2: Driver returns the WebElement reference

t3: Proxy calls click() on the element



└─ If DOM changes between t2-t3, you get StaleElementReferenceException

Imagine you look at a parking spot (findElement), see a car there (get reference), but when you go to open its door (click), someone has locked it using a remote (DOM changed)



Why @FindBy is Flaky

- How PageFactory actually works:
 - The proxy doesn't relocate before every operation – it relocates when the proxy is accessed
 - Multiple operations might share one relocation:

```
// This looks like two operations but may use one relocation:  
element.getText(); // Triggers findElement()  
element.click();  // Might use same reference from getText()
```

1. `element.isDisplayed()` → relocates, returns true
2. DOM updates (e.g., React re-renders)
3. `element.click()` → uses same reference from step 1 → STALE ELEMENT

[Optimizing Page Factory](#)



Interact with elements



clear

- only applies to text fields and content editable elements

```
driver.findElement(By.xpath("//*[@attribute,'value']").clear();
```


Send Keys

Firstly find the element then use `sendKeys(" ")`

```
driver.findElement(By.xpath("//*[@attribute,'value']")).sendKeys("value ");
```

Upload Files

```
<input id="file-upload" type="file" name="file">
```

```
driver.findElement(By.id("id")).sendKeys(path);
```

- If not check hidden input, if not ?

Robot Class (NOT RECOMMENDED)

Click

- applies to any element

```
driver.findElement(By.xpath("//*[@attribute,'value']").click();
```




submit

- **only applies to form elements**

```
driver.findElement(By.xpath("//*[@attribute,'value']")).submit();
```



Information about elements

Is Selected

Firstly find the element then use isSelected();

```
driver.findElement(By.cssSelector("[type=submit]")).isSelected();
```

“

Return true if element(Radio Button/Check Box) for example is selected , otherwise return false

”

Is Displayed

Firstly find the element then use `isDisplayed()`;

```
driver.findElement(By.cssSelector("[type=submit]")).isDisplayed();
```

Is Displayed

This method is used to check if the connected Element is displayed on a webpage. Returns a `Boolean` value, True if the connected element is displayed in the current browsing context else returns false.

This functionality is [mentioned in](#), but not defined by the w3c specification due to the [impossibility of covering all potential conditions](#) 📌. As such, Selenium cannot expect drivers to implement this functionality directly, and now relies on executing a large JavaScript function directly. This function makes many approximations about an element's nature and relationship in the tree to return a value.

[Is It or Is It Not Really Visible; Selenium's Flawed Vision | David Bur - YouTube](#)

Is Enabled

First find the element then use isEnabled();

```
driver.findElement(By.cssSelector("[type=submit]")).isEnabled();
```

“

Returns true if element editable , otherwise return false

”



getText

```
driver.findElement(By.xpath("//*[@attribute,'value']"). getText());
```

[Information about web elements | Selenium](#)

[Selenium: Difference between getDomAttribute\(\) and getDomProperty\(\) | by Aparna M | Mar, 2025 | Medium](#)

Shadow DOM

Shadow DOM

- Shadow DOM is a web standard that enables encapsulation in web components. It allows a part of the DOM tree (called a “shadow tree”) to be hidden from the main document’s DOM. This prevents style and script conflicts, and gives you more modular, reusable components.

```
<custom-element>  
  #shadow-root  
    <p>This is inside shadow DOM</p>  
</custom-element>
```

```
WebElement shadowHost = driver.findElement(By.cssSelector("#shadow_host"));  
    SearchContext shadowRoot = shadowHost.getShadowRoot();  
    WebElement shadowContent = shadowRoot.findElement(By.cssSelector("#shadow_content"));
```

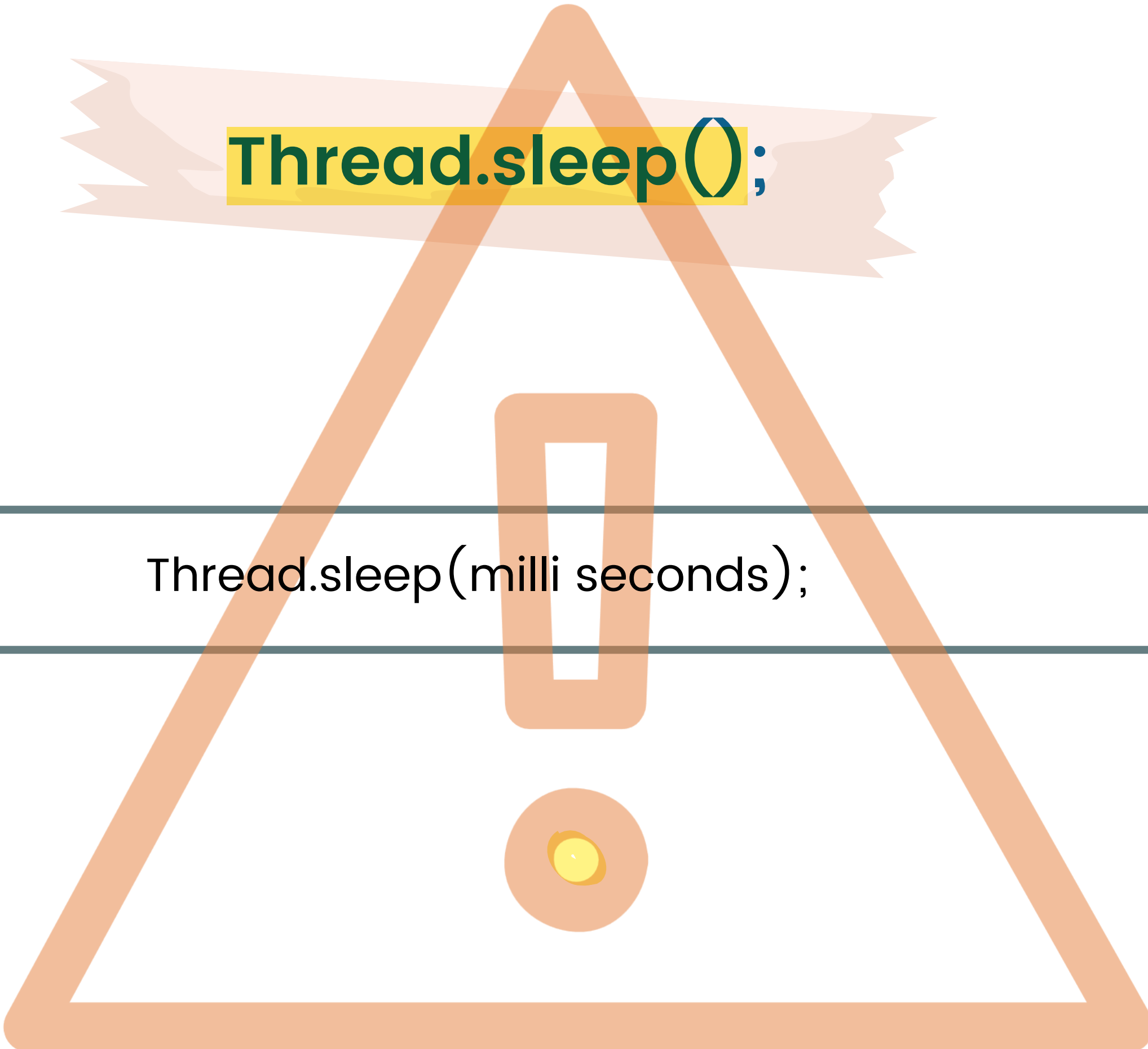

Synchronizations



“

**Selenium doesn't wait if any element
take seconds to be shown in website.**

”



Thread.sleep();

Thread.sleep(milli seconds);

Thread.sleep();

- **Blocking the Thread**
- **Concurrency Issues**
- **Interrupt Handling**
- **Test Automation Challenges**

general

implicitlyWait();

is used to tell the WebDriver to wait for a certain amount of time for the element's presence before throwing a NoSuchElementException

It is set once and then applied throughout the entire session.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(seconds))
```


Disadvantages of implicit Wait



- Only wait if any element takes seconds to be presence in the DOM.
- Don't care if element is visible or is enabled or clickable or ...

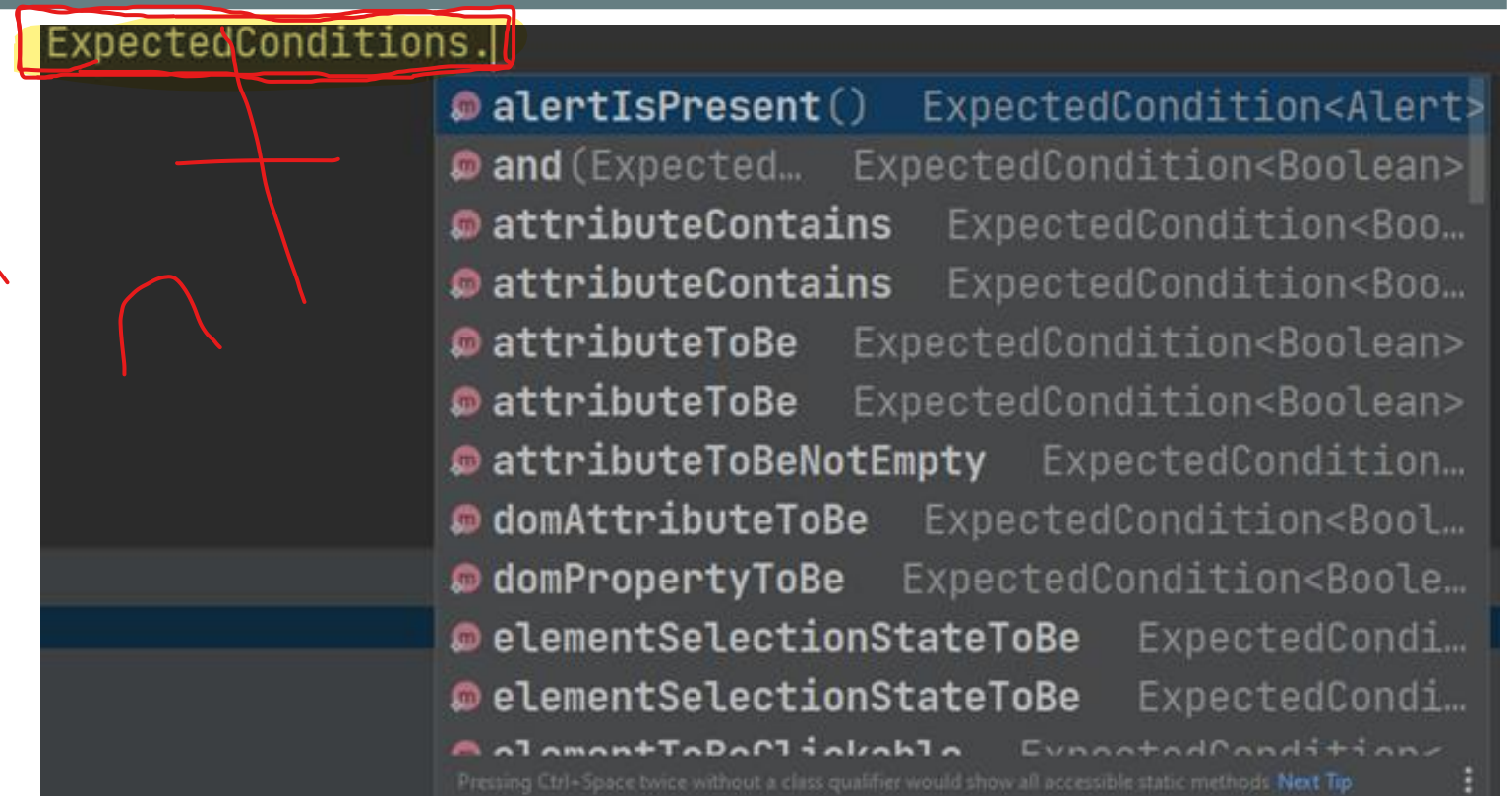
Explicit Wait

Unlike implicit waits, explicit waits are defined for a particular event and specific element.

```
new WebDriverWait(driver,Duration.ofSeconds(10)).until(ExpectedConditions.)
```

Lambda

import



Fluent Wait

Fluent waits are a variation of explicit waits that provide more flexibility in defining wait conditions. They allow you to set polling intervals and ignore specific types of exceptions during the wait also adding custom message.

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(10))
    .pollingEvery(Duration.ofMillis(500)) →
    .ignoring(NoSuchElementException.class);
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("elementId")));
```

check if element is visible

with message

Package Load Strategy

Handling Drop down

Drop Down

Firstly find the element then use Select Class

```
WebElement element = driver.findElement(By.cssSelector("[type=submit]"));
```

- `Select` DropDown = `new Select`(element);

```
DropDown.s
  ◉ selectByIndex(int index) void
  ◉ selectByValue(String value) void
  ◉ selectByVisibleText(String text) void
```

SelectByVisibleText

```
driver.get("https://the-internet.herokuapp.com/dropdown");  
WebElement optionList = driver.findElement(By.id("dropdown"));  
Select options = new Select(optionList);  
options.selectByVisibleText("Option 2");
```

SelectByValue

```
driver.get("https://the-internet.herokuapp.com/dropdown");  
WebElement optionList = driver.findElement(By.id("dropdown"));  
Select options = new Select(optionList);  
options.selectByValue("1");
```

SelectByIndex

```
driver.get("https://the-internet.herokuapp.com/dropdown");  
WebElement optionList = driver.findElement(By.id("dropdown"));  
Select options = new Select(optionList);  
options.selectByIndex(3);
```


getFirstSelectedOption()

```
driver.get("https://the-internet.herokuapp.com/dropdown");  
WebElement options = driver.findElement(By.id("dropdown"));  
  
Select myDrop = new Select(options);  
System.out.println(myDrop.getFirstSelectedOption());
```


Size()

```
driver.get("https://the-internet.herokuapp.com/dropdown");  
WebElement options = driver.findElement(By.id("dropdown"));  
  
Select myDrop = new Select(options);  
System.out.println(myDrop.getOptions().size());
```

WebTables

The main challenge is getting the column, row, or cell that you need

Example 1

No Class or ID attributes to signify groupings of rows and columns

Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete
Conway	Tim	tconway@earthlink.net	\$50.00	http://www.timconway.com	edit delete

WebTables

Use List<> in case you need to loop around the table

```
driver.get("https://the-internet.herokuapp.com/tables");
WebElement myTable = driver.findElement(By.id("table1"));
List<WebElement> rows = driver.findElements(By.tagName("tr"));
System.out.println(rows.size());

for (WebElement row: rows)
{
    List<WebElement> columns = row.findElements(By.tagName("td"));
    for (WebElement column: columns)
    {
        System.out.println(column.getText());
    }
    System.out.println();
}
```

Checkboxes & Radio Button

Checkboxes and Radio Button are treated as a normal element

Checkboxes

- ☐ checkbox 1
☒ checkbox 2

Radio Buttons

Choose your Education level:

- ☐ High School
☐ College
☐ Grad School

```
<form id="checkboxes">  
  <input type="checkbox"> checkbox 1  
  <br>  
  <input type="checkbox" checked>  
    " checkbox 2 "  
</form>
```

```
<div class="col-sm-8 col-sm-offset-2">  
  <input type="radio" id="radio-button-1" aria-label="Radio button"  
    value="radio-button-1" name="education-level">  
    " High School "  
</div>
```



Actions

Double Click

Firstly find the element then use `Action(); doubleClick();`

```
WebElement element=driver.findElement(By.cssSelector("[type=submit]"));
```

```
Actions action=new actions (driver);  
action.doubleClick(element).perform();
```



- **Perform:** for taking the action

Right Click

Firstly find the element then use `Action(); contextClick();`

```
WebElement element=driver.findElement(By.cssSelector("[type=submit]"));
```

```
Actions action=new actions (driver);  
action.contextClick(element).perform();
```

Hovering

Firstly find the element then use `Action(); moveToElement();`

```
WebElement element=driver.findElement(By.cssSelector("[type=submit]"));
```

```
Actions action=new actions (driver);  
action.moveToElement(element).perform();
```

Drag & Drop

Firstly specify Source & Destination the element then use `Action(); dragAndDrop();`

```
WebElement source=driver.findElement(By.cssSelector("[type=submit]"));  
WebElement destination=driver.findElement(By.cssSelector("[type=submit]"));
```

```
Actions action=new actions (driver);  
action.dragAndDrop(source, destination).perform();
```


Click & Hold

Firstly specify Source & Destination the element then use `Action(); clickAndHold(); moveToElement;`

```
WebElement source=driver.findElement(By.cssSelector("[type=submit]"));  
WebElement destination=driver.findElement(By.cssSelector("[type=submit]"));
```

```
Actions action=new actions (driver);  
action.clickAndHold(source).moveToElement(destination).release().build().perform();
```

Keys

```
driver.findElement(By.id("id")).sendKeys(Keys.);
```

Ⓢ ENTER	Keys
Ⓢ END	Keys
Ⓜ chord(CharSequence... value)	String
Ⓢ ADD	Keys
Ⓜ getKeyFromUnicode(char key)	Keys
valueOf(String name)	Keys
values()	Keys[]
Ⓜ chord(Iterable<CharSequence> value)	String
Ⓢ ALT	Keys
Ⓢ ARROW_DOWN	Keys
Ⓢ ARROW_LEFT	Keys
Ⓢ ARROW_RIGHT	Keys

Keys in Actions

```
Actions actions = new Actions(driver);  
actions.
```

```
Ⓜ keyDown(CharSequence key)      Actions  
Ⓜ sendKeys(CharSequence... key... Actions  
Ⓜ keyDown(WebElement target, C... Actions  
Ⓜ keyUp(CharSequence key)        Actions  
Ⓜ keyUp(WebElement target, Cha... Actions
```

Scrolling in Actions

```
Actions actions = new Actions(driver);  
actions.
```

```
    scrollToElement(WebElement element) Actions  
    scrollByAmount(int deltaX, int deltaY) Actions  
    scrollFromOrigin(ScrollOrigin origin) Actions
```

Handling Windows

getWindowHandle();

To get the Context unique identifier of the current page.

```
String WindowId = driver.getWindowHandle();
```

getWindowHandles();

To get the Context unique identifier of the all pages you clicked in set {p1,p2,p3,p...}.

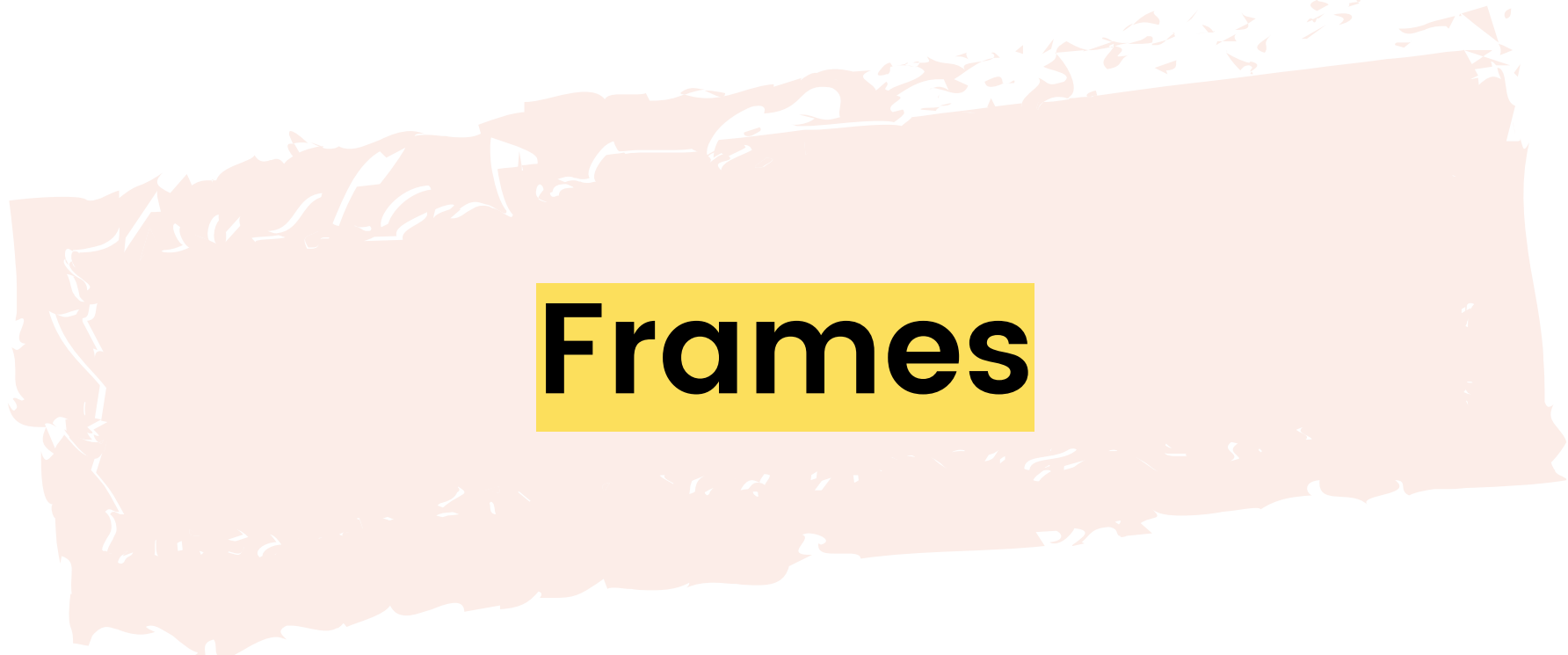
```
Set<string> allWindows = driver.getWindowHandle();
```



switchTo();

To switch between windows

```
driver.switchTo().window(window);
```

Frames

frame();

GeeksforGeeks
A computer science portal for geeks

GeeksforGeeks
A computer science portal for geeks

GeeksforGeeks
A computer science portal for geeks

efficient and successful. Placement preparation solely depends on the company for which you are preparing. There are basically three different categories into which we can divide the companies visiting campuses for placements based on their recruitment process. Mass Recruiters, Tech



frame();

```
driver.switchTo().frame("name");
```

frame();

```
driver.switchTo().frame("id");
```



frame();

```
Web element = driver.findElement(By.id("id"))
```

```
driver.switchTo().frame("web element");
```

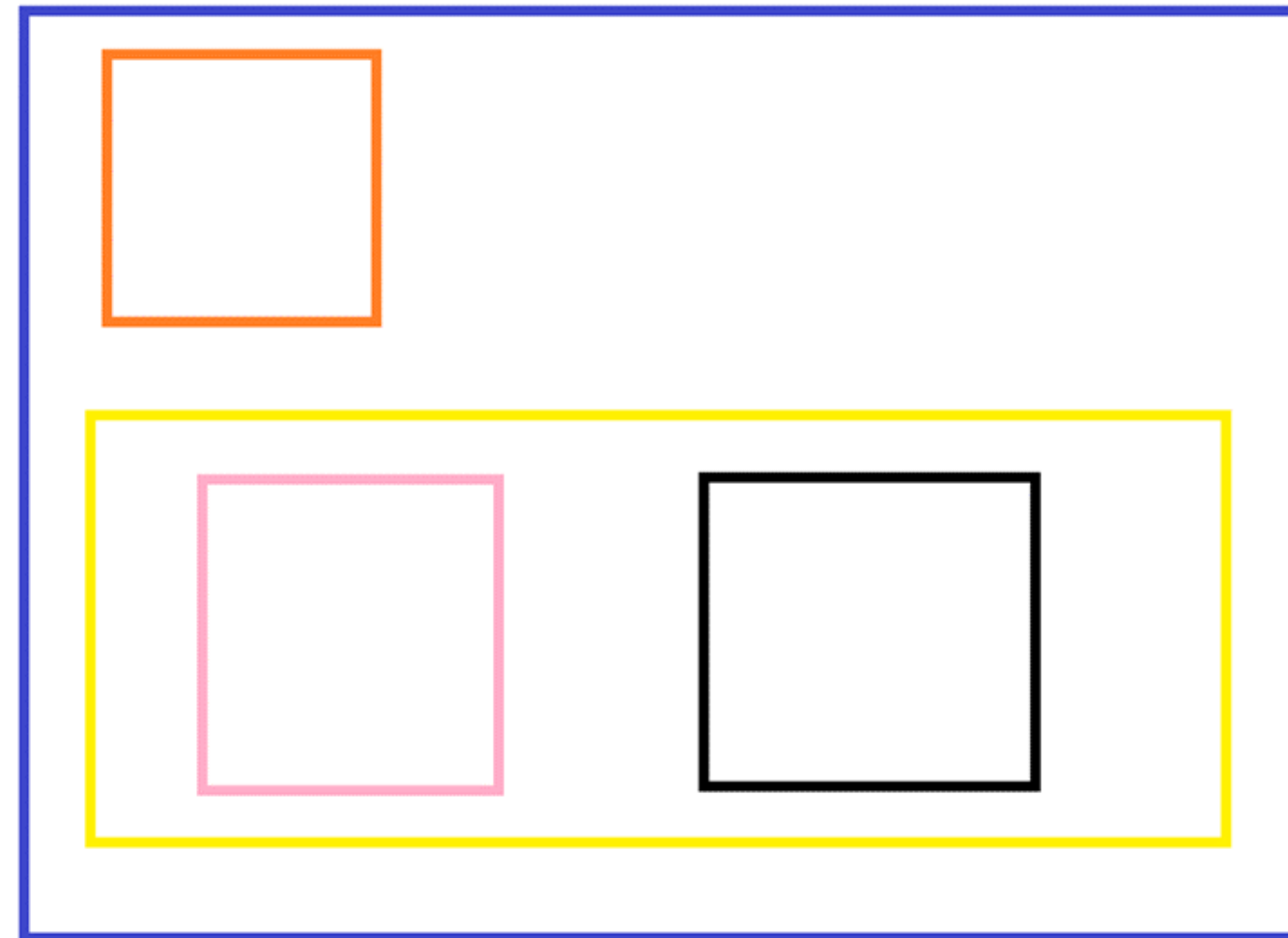



```
parentFrame();
```

After finishhigh your work in any frame you need to switch to parent frame.

```
driver.switchTo().parentFrame()
```

Nested Frames





Alerts

alert();

```
driver.findElement(By.id("id"))
```

```
driver.switchTo().alert().accept();
```

Get text from alert();

```
driver.switchTo().alert().getText();
```

Send keys to alert();

```
driver.switchTo().alert().sendKeys("Value");
```

Accept alert();

```
driver.switchTo().alert().accept();
```

Dismiss alert();

```
driver.switchTo().alert().dismiss();
```


Advanced Topics in selenium

Taking Screenshots

```
File src = ((TakesScreenshot)driver).getScreenshotAs(OutputType.FILE);
```

```
File dest = new File("src/main/resources/test.png");
```

```
FileUtils.copyFile(src,dest);
```

Browser Options

```
edgeOptions = new EdgeOptions();  
HashMap<String,Object> pref = new HashMap<>();  
pref.put("download.prompt_for_download",false);  
pref.put("download.default_directory","path");  
edgeOptions.setExperimentalOption("prefs",pref);  
driver = new EdgeDriver(edgeOptions);
```

- for more info

<https://chromedriver.chromium.org/capabilities>

Check broken links

```
public static void checkLinkBroken(By element) {  
    String href = driver.findElement(element).getDomAttribute("href");  
    URL url = new URI(href).toURL();  
    Response response = RestAssured.given().get(url);  
    System.out.println("Link response: " + response.getStatusLine());  
}
```

Check broken Images

```
public static void checkImgBroken(By element) {  
    String href = driver.findElement(element).getDomAttribute("src");  
    URL url = new URI(href).toURL();  
    Response response = RestAssured.given().get(url);  
    System.out.println("Link response: " + response.getStatusLine());  
}
```

JavaScript Executor

JavaScript Executor

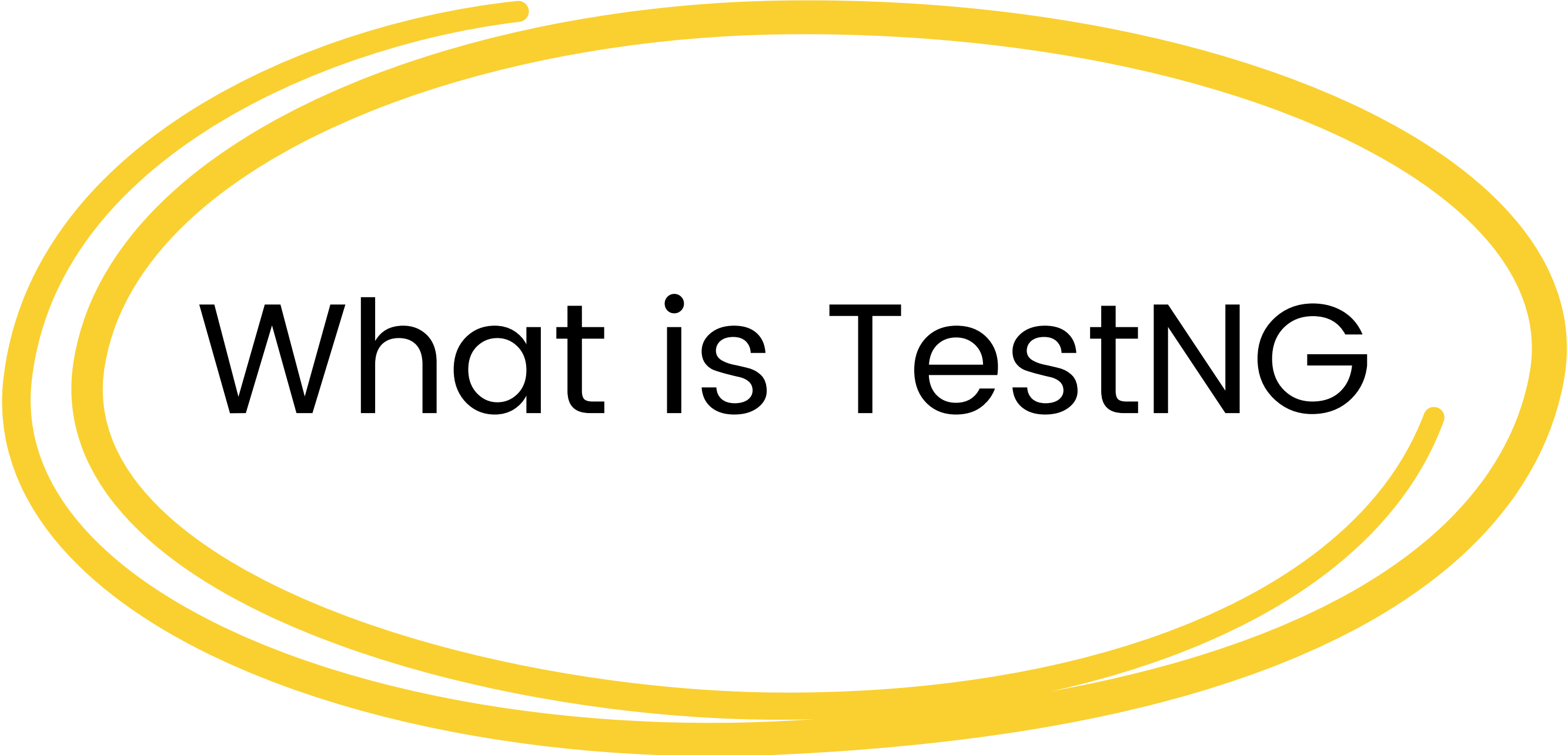
- **JavaScriptExecutor** is an interface provided by Selenium that helps in executing JavaScript commands
- Disadvantage:
 - **Force Injection** instead of simulating real user behaviour

[QA & Testing: Why using JavascriptExecutor in WebDriver can be dangerous](#)

Scrolling using JS

```
((JavascriptExecutor)driver).  
executeScript("arguments[0].scrollIntoView();",driver.findElement(By.name("uid")));
```


Testing Framework (TestNG)

A thick yellow double-oval line highlights the text "What is TestNG".

What is TestNG

JUnit + TestNG → JAVA

NUnit → C#

What is TestNG

ایک نیا ٹیسٹ فریم ورک
جس میں کئی نئی چیزیں
ہیں۔

- TestNG is a testing framework introducing some new functions that make automation more powerful and easier to use.

TestNG

- TestNG is designed to cover all categories of tests: unit, functional, end-to-end, integration, etc....

Core functions of a Test Framework

- Create & Execute Test Scripts
- Listeners and Generate Test Reports
- Flexible Annotations
- Parameterized Testing
- Parallel Execution
- Grouping of Tests

TestNG



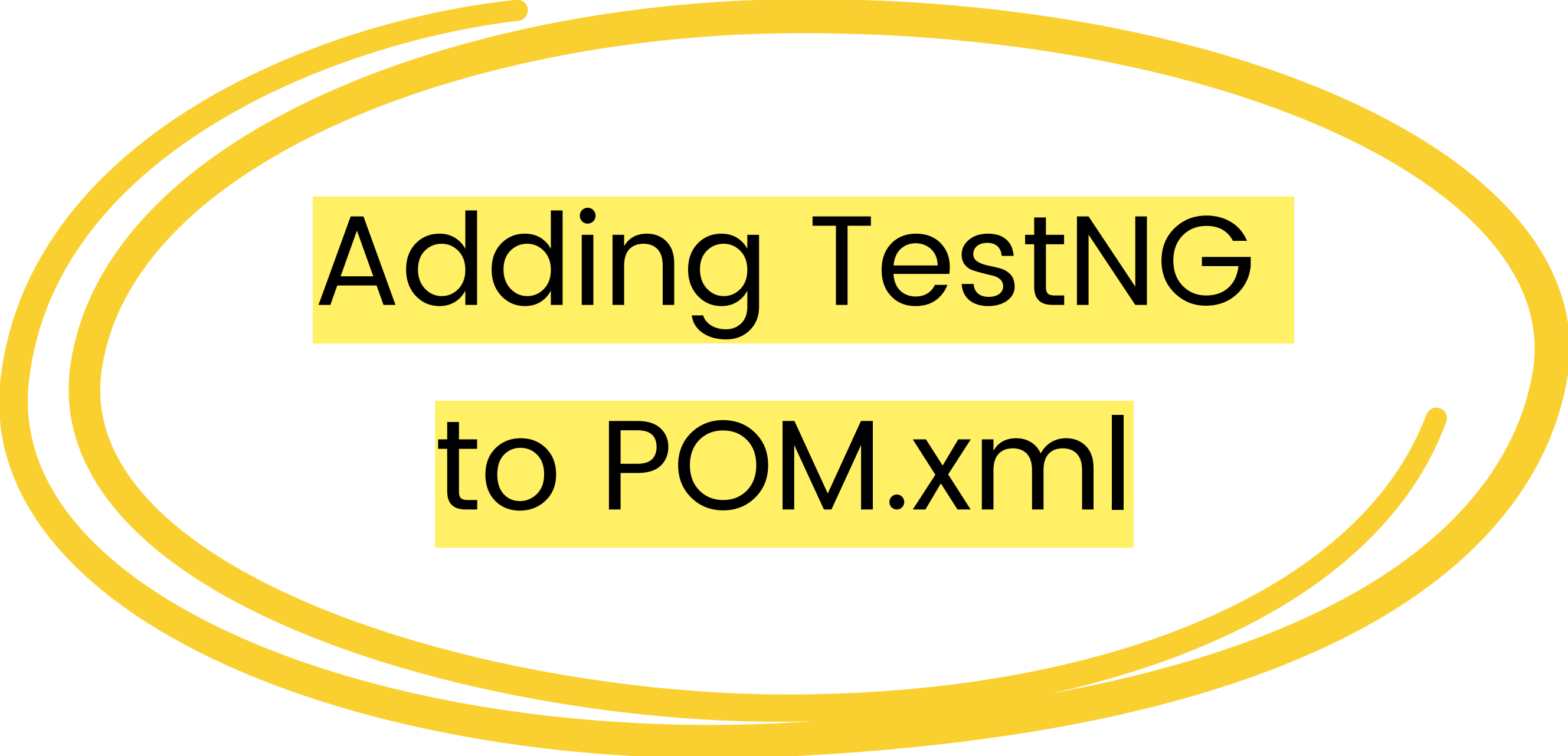
Why TestNG

Why TestNG

- TestNG in Selenium provides an option `testng-failed.xml`, If you want to run only failed test cases.
- Each test case executed and evaluated separately.
- Create testing suite file where Multiple test cases can be grouped more easily by converting them into `testng.xml` file.

Why TestNG

- Can **make priorities** which test case should be executed first.
- Parallel testing is possible using TestNG.
- Parametrization provided by TestNG is more convenient and easier to use through data provider



Adding TestNG
to POM.xml

Adding TestNG to POM.xml

1-Go to:

<https://mvnrepository.com>

2-Go to the latest version stable

3-Add dependency in pom.xml file

TestNG

TestNG » 7.9.0
TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionalities that make it more powerful and easier to use. It supports test configured by annotations, data-driven testing, parametric tests, etc.

License	Apache 2.0
Categories	Testing Frameworks & Tools
Tags	testing testng quality
HomePage	https://testng.org
Date	Dec 26, 2023
Files	pom (2 KB) jar (1011 KB) View All
Repositories	Central
Ranking	#51 in MvnRepository (See Top Artifacts) #6 in Testing Frameworks & Tools
Used By	11,459 artifacts

Maven

Gradle

Gradle (Short)

Gradle (Kotlin)

SBT

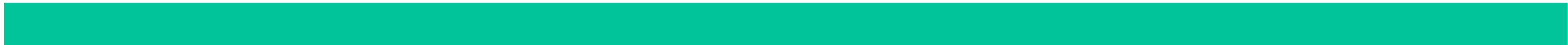
Ivy

Grape

Leiningen

Buildr

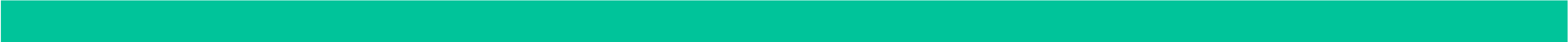
```
<!-- https://mvnrepository.com/artifact/org.testng/testng -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.9.0</version>
  <scope>test</scope>
</dependency>
```





TestNG

Annotations



TestNG Annotations



An annotation is a tag that provides information about the method, class, and suite. It helps to define the execution approach of your test cases and the different features associated with it.



Pre-conditions

@BeforeSuite: run before all tests in this suite have run.

يكون دى حاجة آخره ال suite

@BeforeTest: run before any test method belonging to the classes inside the tag is run.

@BeforeClass: runs before the execution of test methods in a current class.

@BeforeMethod: annotated method will be run before each test method.

دو بيتا method مدلو بين في جروب

@BeforeGroup: annotated method will be run before each group.

دو Test Case

Post-conditions

@AfterClass: run after all the test methods in the current class have been run.

@AfterSuite: run after all tests in this suite have run.

@AfterTest: run after all the test methods belonging to the classes inside the tag have run.

@AfterMethod: annotated method will be run after each test method

@AfterGroup: annotated method will be run after each group.

Before and After method ال test دائما وابدأ يبقو محاطين ال testcase الي تخصصهم لو في عندي 100 ف test

Method

Test 1

After Method

Before Method

Test 2

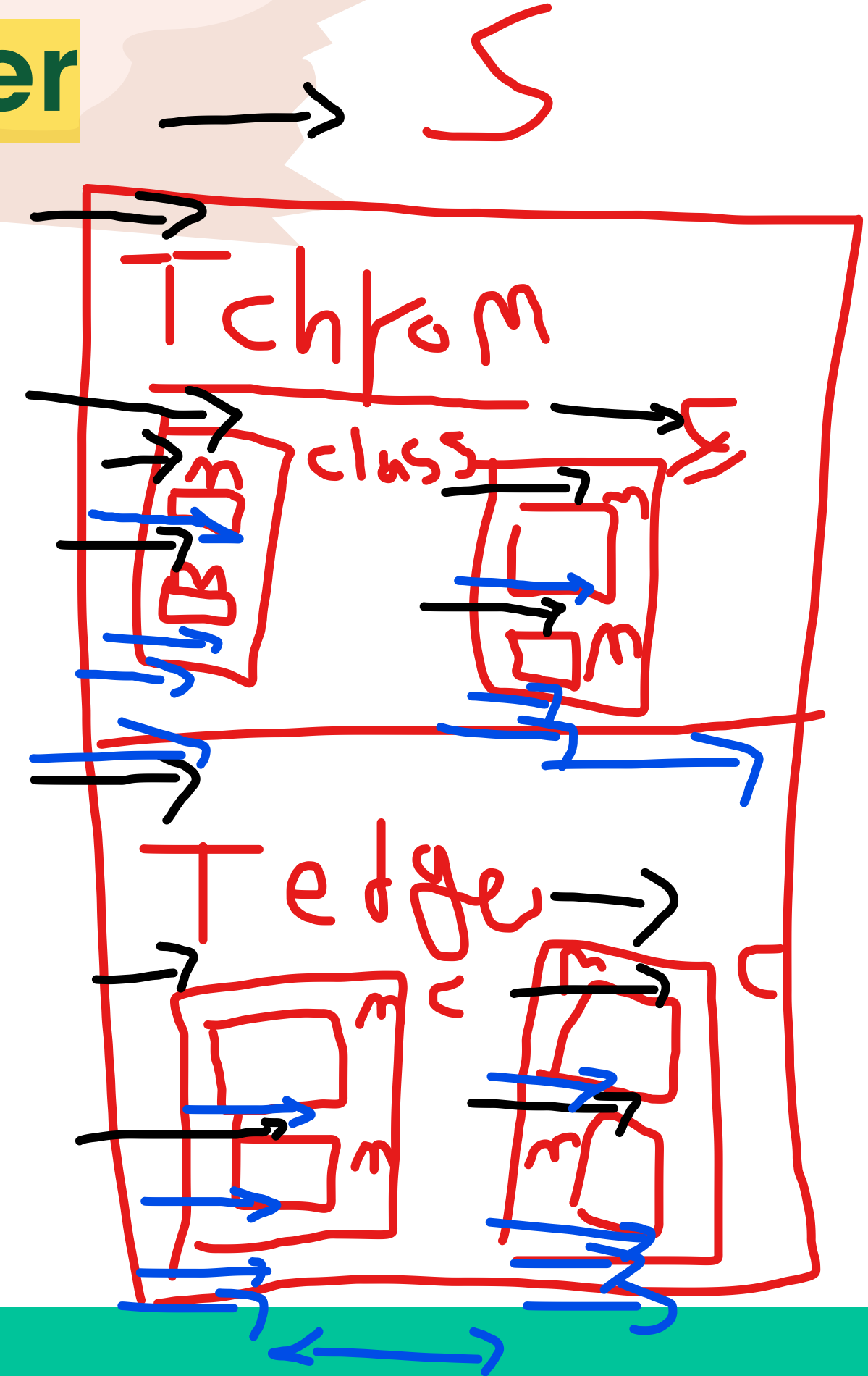
After Method

Execution Order

very

important

1. Before Suite
2. BeforeTest
3. BeforeClass
4. BeforeMethod
5. Test
6. AfterMethod
7. AfterClass
8. AfterTest
9. AfterSuite



Class Level

@Test: The effect of a class level @Test annotation is to make all the public methods of this class to become test methods even if they are not annotated. You can still repeat the @Test annotation on a method if you want to add certain attributes..

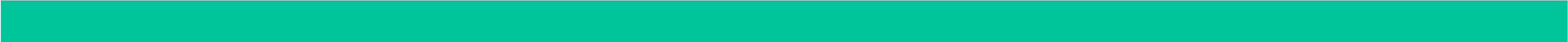
```
@Test
public class Test1 {
    public void test1() {
    }

    public void test2() {
    }
}
```



TestNG

Assertions



TestNG Assertions

- TestNG provides multiple level assertions to validate your actual results against your expected results.
- If the actual result is equal to expected result: Pass
- If the actual result is not equal to expected result: Fail

TestNG Assertions

- **assertTrue**: Verifies whether the defined condition is true or not. If true, it will pass the test case. If not, it will fail the test case

```
Assert.assertTrue(condition, Message);
```

Optional

- **assertFalse**: Verifies whether the defined condition is false or not unlike assertTrue.

```
Assert.assertFalse(condition, Message);
```

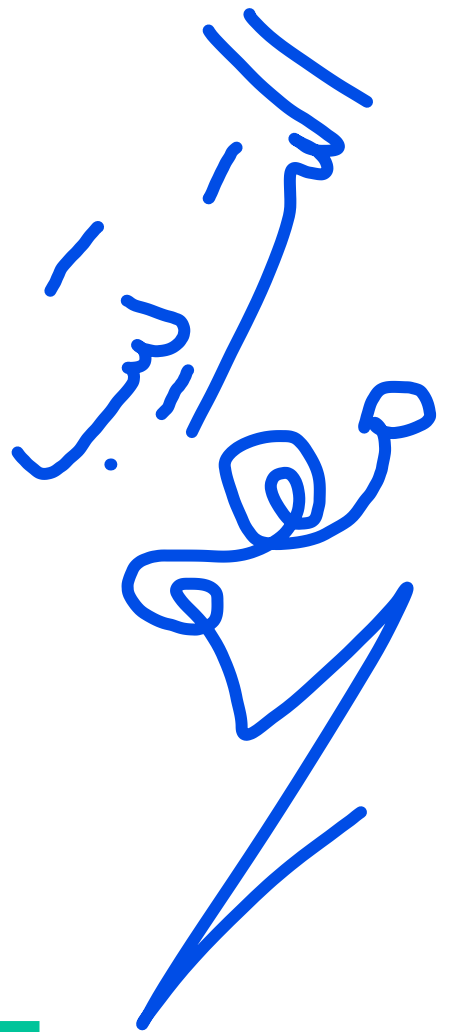
TestNG Assertions

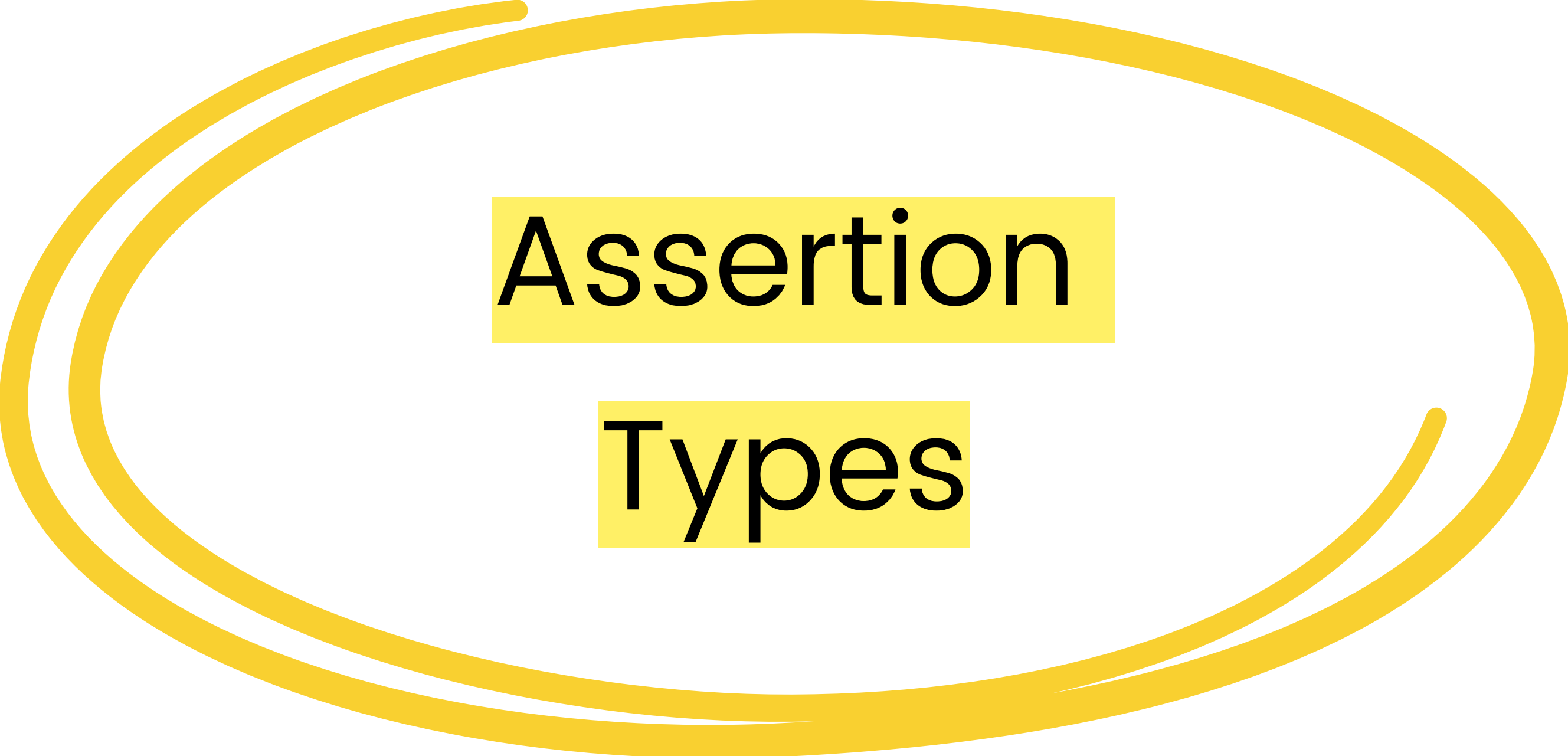
- **assertEquals:** Compares the expected value with actual value. If both are the same, it passes the test case. If not, it fails the test case.

```
Assert.assertEquals(actual, expected, Message(optional) );
```

- **assertNotEquals:** It's just opposite to what assertEquals does

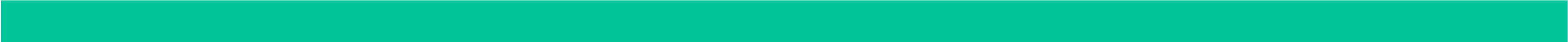
```
Assert.assertNotEquals(actual, expected, Message(optional) );
```





Assertion

Types



Assertion Types

```
graph TD; A[Assertion Types] --> B[Hard Assertion]; A --> C[Soft Assertion];
```

Hard Assertion

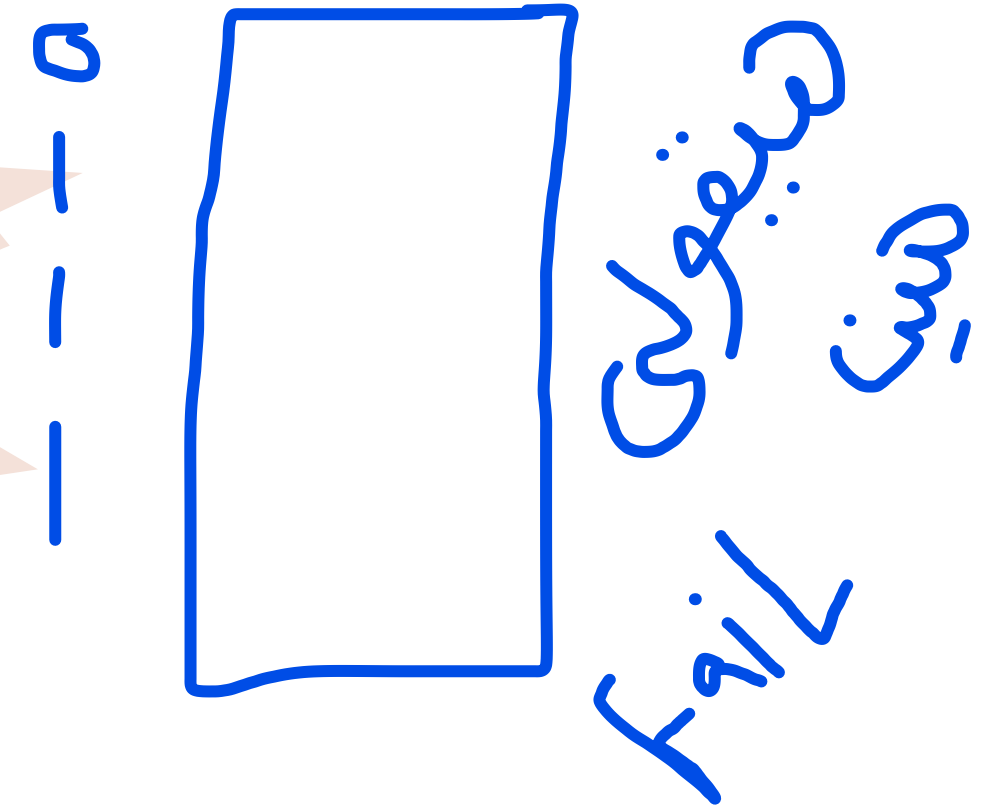
Soft Assertion

Hard Assert

If assert condition gets failed in Hard Assert :

- TestNG throws an exception immediately after the assert fails
- The remaining statements inside the method will be aborted.
- TestNG carries out with the next test case of the suite.

Soft Assert



If assert condition gets failed in Soft Assert:

- TestNG collects errors during execution and doesn't throws an exception
- The remaining statements inside the method will be executed
- If there is any exception and you want to throw it then you need to use assertAll()
- TestNG will carry out with the next test case of the suite.



TestNG Priority

TestNG Priority

→ Testing
بجایزه

severity

Prioritization in TestNG is a way to provide a sequence to the methods so that they do not run out of order.

→ reports
و
Allure
پیشرفت

TestNG Priority

- Priority in TestNG test cases is a parameter with attribute value

E.X: `@Test(priority=1)`

- If no priority mentioned, Testng will execute the @Test methods based

Alphabetical order of their method names irrespective of their place of implementation in the code.

Enabled

- Enable or disable any class/method

E.X: `@Test(enabled=true)`

disable = True

Ignore

- disable any class/method

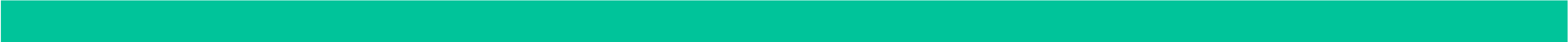
```
@Ignore
public class ParametersClass {
    |
}
```

```
@Ignore
@Test(priority = 3)
public void test1 ()
{
    System.out.println("Ahmed");
}
```



TestNG

Dependencies



Method Dependencies

In case you want to execute function only if another function executed successfully we can do it using `dependsOnMethod()`;

```
@Test
public void Register()
{
    System.out.println("User Registered Successfully");
}

@Test (dependsOnMethods = "Register")
public void Login()
{
    System.out.println("User Logged-in Successfully");
}
```

Method Dependencies

©So in this case if Register “pass” , Login will be executed but if “fails” Login will be skipped

```
@Test
public void Register()
{
    System.out.println("User Registered Successfully");
}

@Test (dependsOnMethods = {"Register "})
public void Login()
{
    System.out.println("User Logged-in Successfully");
}
```

Method Dependencies

Incase you need to make the test case always run even it's dependent method fails, you can use `@always run` annotation

```
@Test
public void register()
{
    // System.out.println("Register Test Case");
    throw new RuntimeException();
}

@Test(dependsOnMethods = {"register"},alwaysRun = true)
public void login()
{
    System.out.println("Login Test Case");
}
```




Dealing with
.xml file

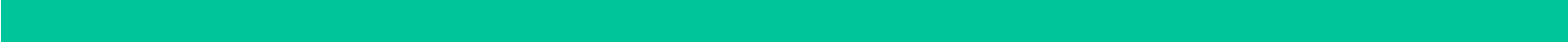
Dealing with .xml file

.xml file works as test runner plus other features as:

- Grouping
- Parameterization

A large, hand-drawn style yellow oval with a thick stroke, centered on the page. It encloses the text 'TestNG Groups'.

TestNG Groups

A solid teal horizontal bar at the bottom of the slide.

Grouping

- Suppose that you have a 50 test case and if you want to run only 20 of them, what you will do ?
- Exactly you can do it using grouping You can assign a name of group to test case , then later you run this group instead of running test cases

Grouping

```
@Test(groups = {"Regression"})
public void Register()
{
    System.out.println("User Registered Successfully");
}

@Test (groups = {"Regression"})
public void Login()
{
    System.out.println("User Logged-in Successfully");
}
```

Grouping

Now you need to edit your testing.xml file and add Groups tag and add the name of group you create

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="All Test Suite">
  <test name="TestTest">
    <groups>
      <run>
        <include name="Regression"> </include>
      </run>
    </groups>
    <classes>
      <class name="TestTest"/>
    </classes>
  </test>
</suite>
```

Grouping

What about @before and @after methods ?

- You can include your @before and @after in your group, or
- Make it alwaysRun=true;

Group of groups

Groups can also include other groups. These groups are called "MetaGroups".

```
<test name="Regression1">
  <groups>
    <define name="functionalTest">
      <include name="edge"/>
      <include name="chrome"/>
    </define>

    <run>
      <include name="functionalTest"/>
    </run>
  </groups>

  <classes>
    <class name="test.sample.Test1"/>
  </classes>
</test>
```


Partial groups

You can define groups at the class level and then add groups at the method level

```
@Test(groups = {"Regression Test","broken"})
public void testCase1()
{
    System.out.println("Test Case 1");
    // throw new RuntimeException();
}
```

```
@Test(groups = { "checkin-test" })
public class All {

    @Test(groups = { "func-test" })
    public void method1() { ... }

    public void method2() { ... }
}
```

Exclusion groups

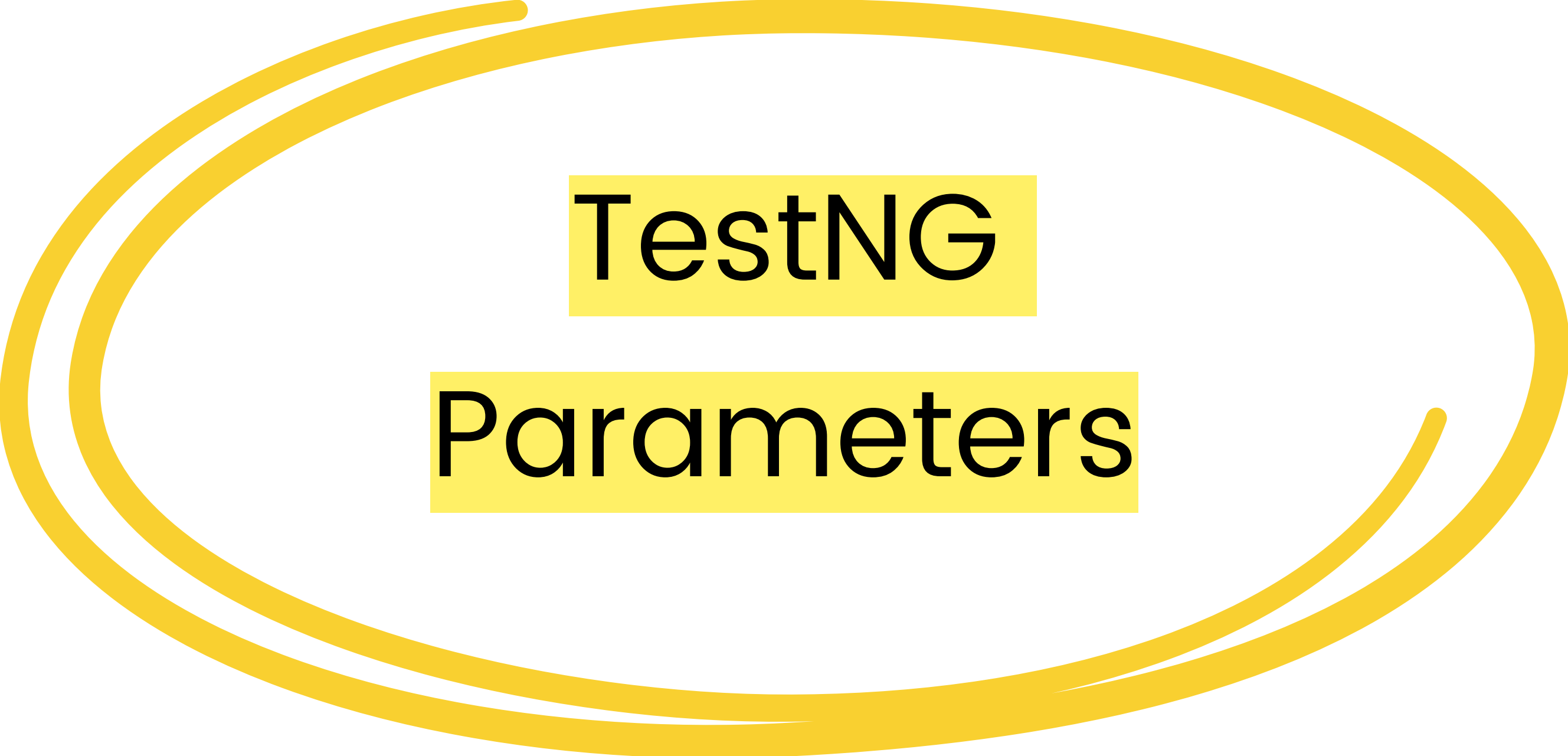
TestNG allows you to include groups as well as exclude them.

- For example, it is quite usual to have tests that temporarily break because of a recent change, and you don't have time to fix the breakage yet

```
@Test(groups = {"Regression Test", "broken"})
public void testCase1()
{
    System.out.println("Test Case 1");
    // throw new RuntimeException();
}
```

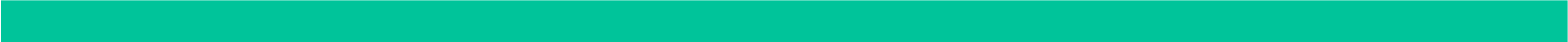
```
<test name="Simple example">
  <groups>
    <run>
      <include name="checkintest"/>
      <exclude name="broken"/>
    </run>
  </groups>

  <classes>
    <class name="example1.Test1"/>
  </classes>
</test>
```



TestNG

Parameters



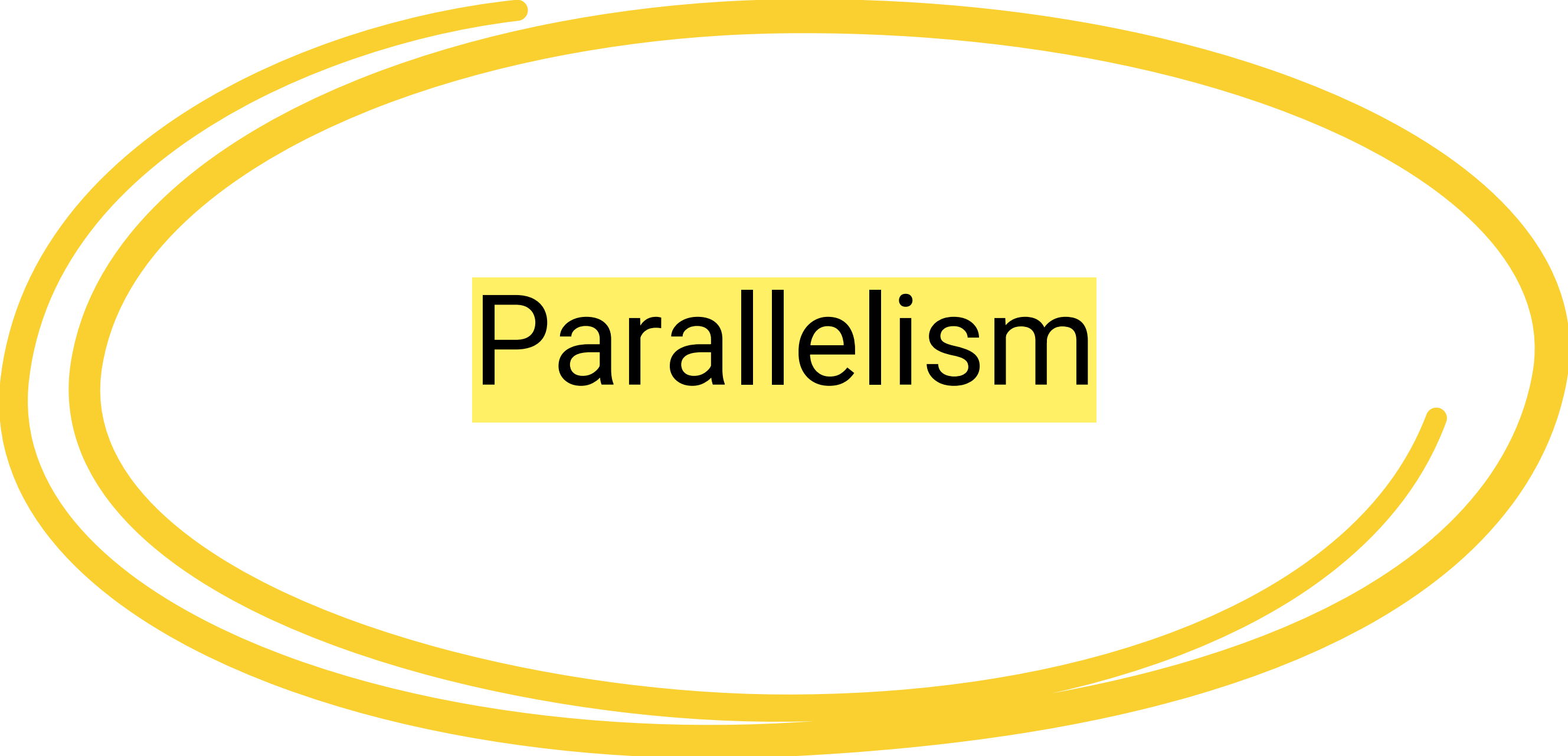
Parameters

Test methods don't have to be parameterless. You can use an arbitrary number of parameters on each of your test method, and you instruct TestNG to pass you the correct parameters with the `@Parameters` annotation.

```
@Parameters({"username","password"})
@Test()
public void validLoginTestCase (@Optional("tomsmith") String username,@Optional("SuperSecretPassword!") String password)
```

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="Test" >
  <parameter name="username" value="tomsmith"/>
  <parameter name="password" value="SuperSecretPassword!"/>
```

- If no parameter named “**username,password**” is found in your **testng.xml** file, your test method will receive the default value specified inside the **@Optional** annotation.



Parallelism

Parallelism

You can instruct TestNG to run your tests in separate threads in various ways.

- You can do it in testing.xml

```
<suite name="My suite" parallel="methods" thread-count="1" time-out= "10000">  
<suite name="My suite" parallel="tests" thread-count="5" time-out= "10000">  
<suite name="My suite" parallel="classes" thread-count="5" time-out= "10000">  
<suite name="My suite" parallel="instances" thread-count="5" time-out= "10000">
```

- You can do it in @Test parameter

```
@Test(threadPoolSize = 2, invocationCount = 2, timeOut = 10000)  
public void test ()  
{  
  
}
```

timeOut works in both parallel and non-parallel.

Thread Local

ThreadLocal is a class in Java that provides thread-local variables. Each **thread** can **independently** access **its copy** of the ThreadLocal variable

Set

Get

Remove



Listeners

Listeners

There are several **interfaces** that allow you to modify TestNG's behavior. These interfaces are broadly called "**TestNG Listeners**".

- **InvokedMethodListener**
- **ITestListener**
- **ISuiteListener**

IExecutionLi →

```
<suite>
<listeners>
  <listener class-name="com.example.MyListener" />
</listeners>
...
```

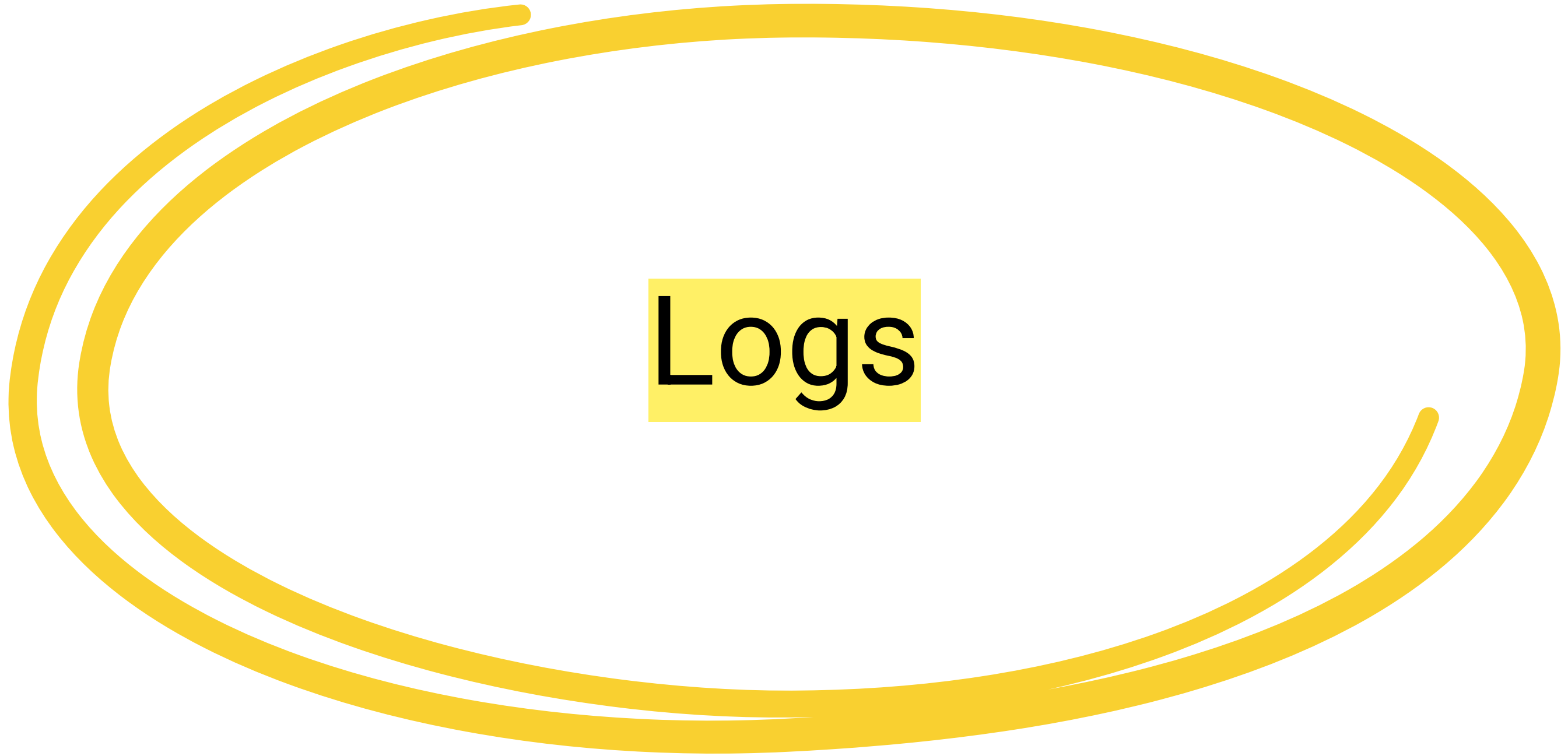
```
@Listeners(ITestListener.class)
public class TestClass {
}
```

Retry Analyzer

Sometimes, you might want TestNG to automatically **retry a test whenever it fails**. In those situations, you can use a retry analyzer.

- **IRetryAnalyzer**

```
@Test(retryAnalyzer = MyRetry.class)
public void test2() {
    Assert.fail();
}
```



Logs

Logs

Logging is an essential part of software development and operations because it helps to understand what's happening in the application, **diagnose issues**, **monitor performance**, and **troubleshoot problems**.

- **SLF4J** is an **abstraction layer** or facade, providing a common API for various logging frameworks. It does not provide its own implementation but serves as a bridge to other logging frameworks as: **Log4j**, **Logback**, **Java Util Logging (JUL)**, and others.

٧٢

هي مجرد وسيط يتاخذ ال logs من مكان ما هي جايه تديها لل
دي فين في فايل logs بتاعتك وانت تشوف عايز تطبع ال framework
او في ال console ف اي مكان

Logs Levels

- **TRACE:** Detailed information for debugging, with the most fine-grained details.
- **DEBUG:** Debugging information providing insight into application behavior during runtime.
- **INFO:** Informational messages indicating important events or application state changes.
- **WARN:** Warnings for potential issues or unexpected but non-fatal situations.
- **ERROR:** Error messages for unexpected conditions that prevent normal application operation.
- **FATAL:** Critical errors indicating severe failures leading to application termination or system breakdown.

Log4j2

- You should add its dependency plus SLF4J dependencies for **API** and **Simple**
- You should configure it using either **.xml** or **.properties** file

- Reference:

- [Log4j – \(apache.org\)](http://log4j.apache.org)

```
# Log files location
property.basePath=test-outputs/Logs

appender=file, stdout

# Define the file appender
appender.file.type=File
appender.file.name=fileAppender
appender.file.fileName=${basePath}/log_${date:yyyy-MM-dd_hh-mm-ss a}.log
appender.file.layout.type=PatternLayout
appender.file.layout.pattern=[%level] %d{dd-MM-yyyy HH:mm:ss} [%t] [%c] - %msg%n

# Define the stdout appender
appender.stdout.type=Console
appender.stdout.name=consoleAppender
appender.stdout.layout.type=PatternLayout
appender.stdout.layout.pattern=[%level] %d{dd-MM-yyyy HH:mm:ss} [%t] [%c] - %msg%n

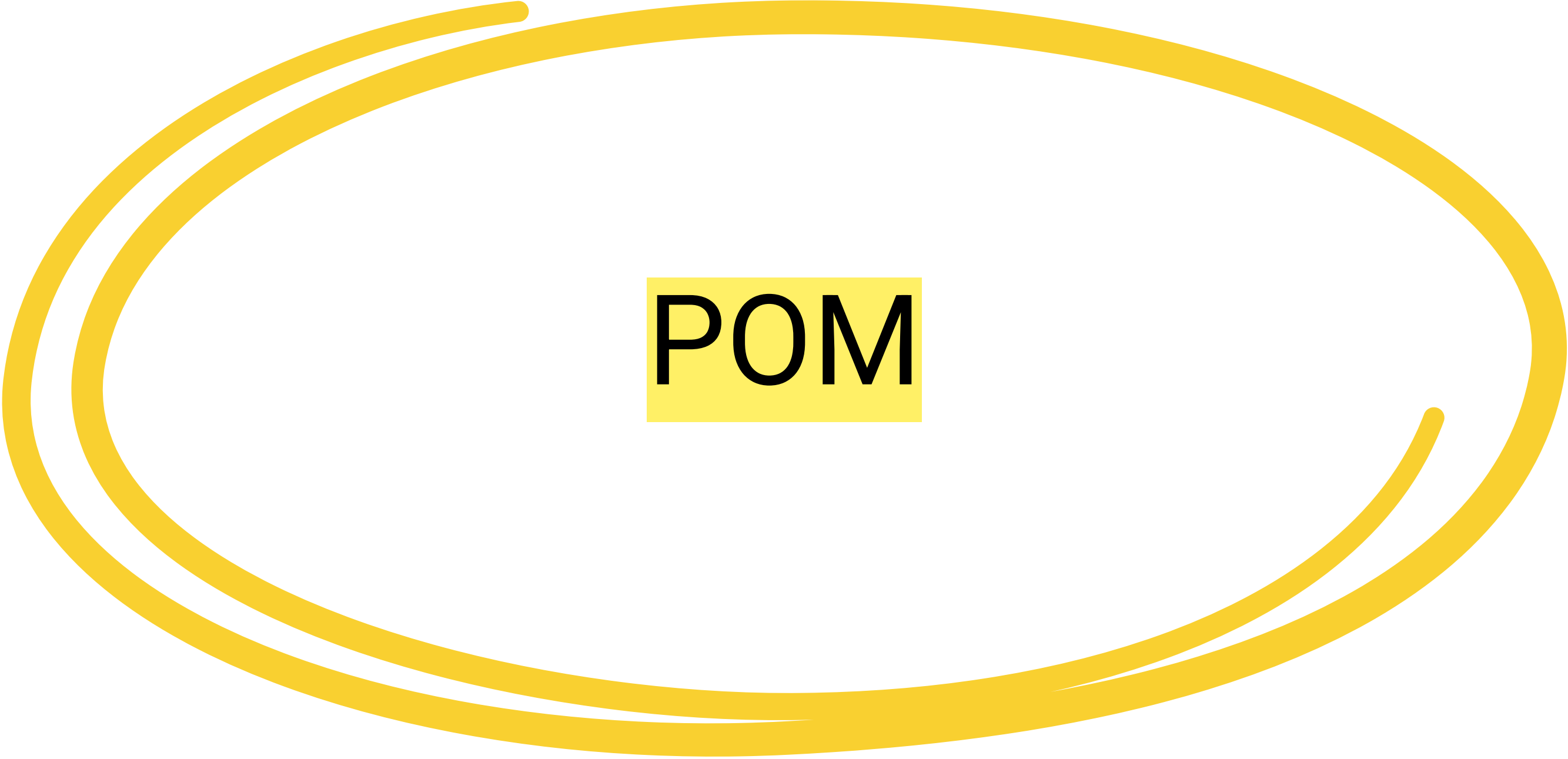
# Configure root logger
rootLogger.level=TRACE
rootLogger.appenderRef.file.ref=fileAppender
rootLogger.appenderRef.stdout.ref=consoleAppender
```

Log4j2

```
public static Logger LOGGER = LogManager.getLogger();
```

//Thread.currentThread().getStackTrace()[2].toString(): for getting the location that the logger called in.

```
public static Logger LOGGER = LogManager.getLogger(Thread.currentThread().getStackTrace()[2].toString());
```

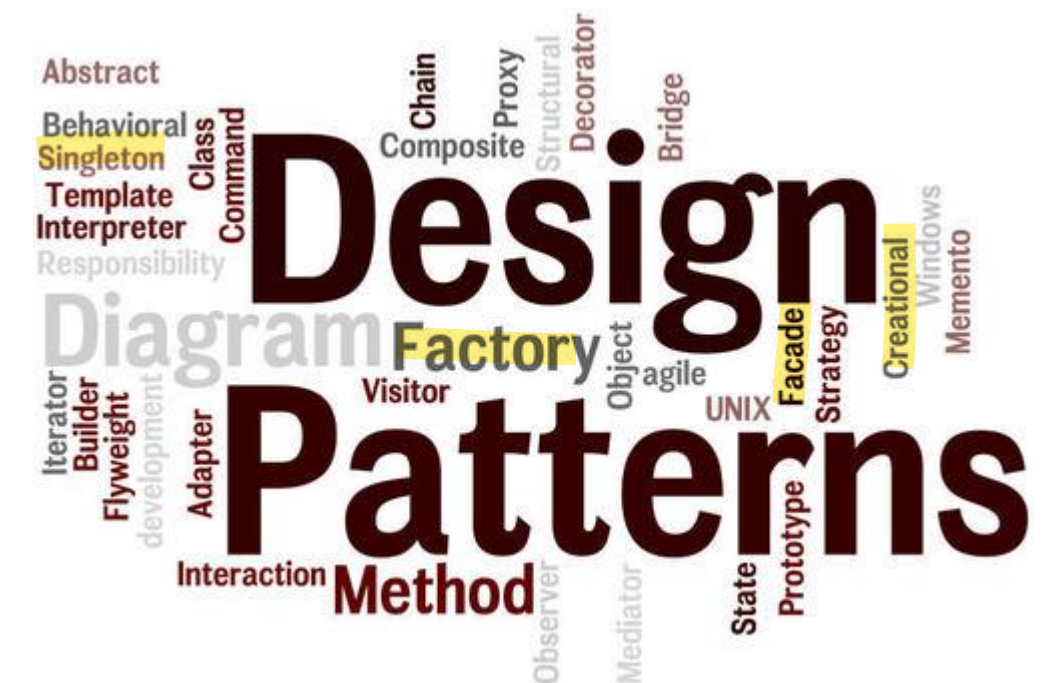
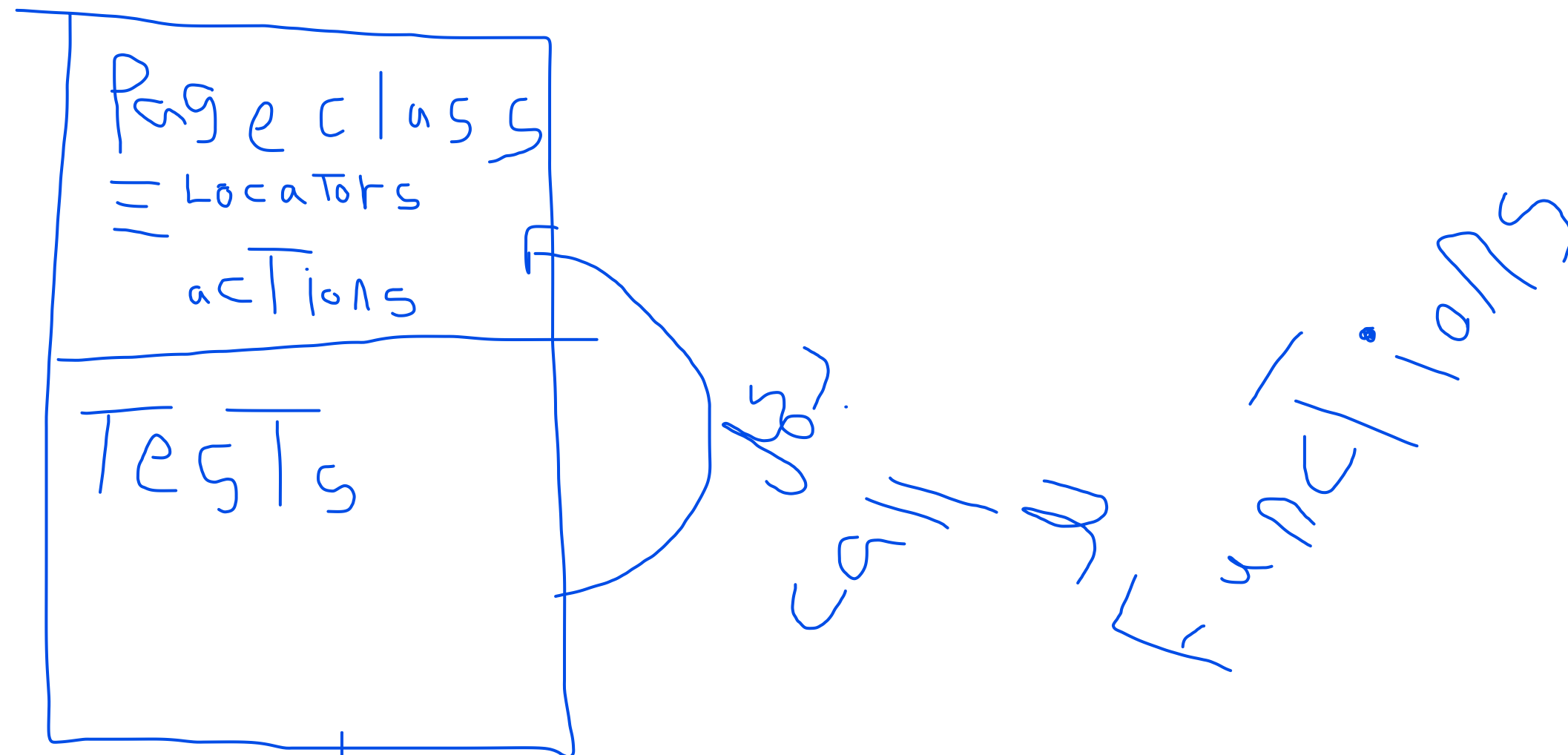


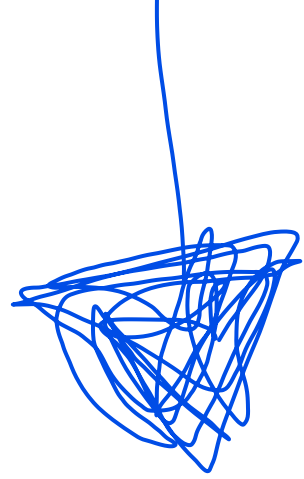
POM

Design Patterns

بيقلل ال duplicated code و بيخلي اني اعمل maintain لاي حاجه الموضوع ييقي سهل

A design pattern provides a **general reusable solution** for the **common problems** that occur in software design, and other developers who've been down the same design problem road and survived the trip.





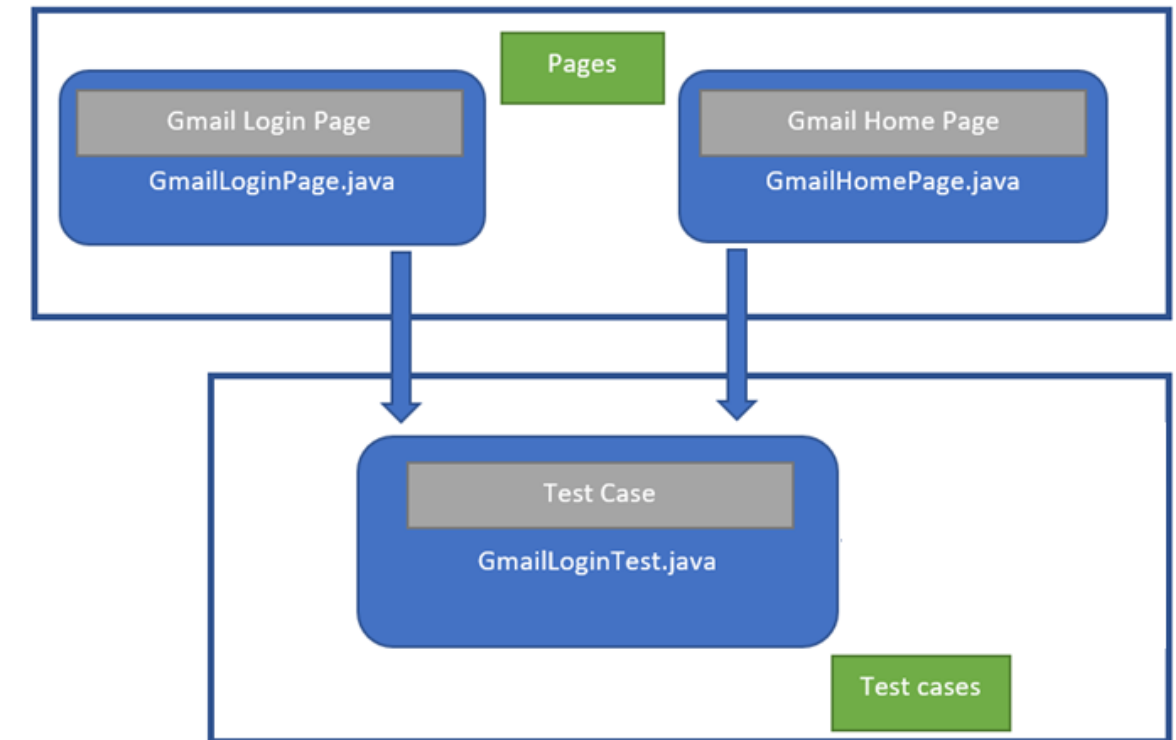
Page Object Model

Why Page Objects Pattern (POM) ?

- Scripts Maintenance
- When some locator changes you don't need to change every piece of code use it but only change it in its page class
- Problem duplicated code increase the cost of maintenance

Page Object Model

- The Page object is an object-oriented class which acts as an interface for the page of your Application under test.
- Page class contains web elements and methods to interact with web elements.
- While Automating the test cases, we create the object of these Page Classes and interact with web elements by calling the methods of these classes.



Page Object Model

Advantages The Page Objects Model (POM)

- **Reusability**: We can reuse the page class if required in different test cases which means we don't need to write code for identifying the web elements and methods to interact with them for every test case.
- **Maintainability**: As we can see from the above picture test case and page class are different from each other which means we can easily update the code if any new web element is added or existing one updated.
- **Readability**: As we can see in the above picture page code is separated from test code which helps to improve code readability.

Page Object Model

Dis-Advantages The Page Objects Model (POM)

- Complexity is increased
- Applying design pattern incorrect will make framework complex to use and maintenance



Your Turn !

10 actionable test automation tips from the experts featured in the "10 Quick Actionable Test"

1. Use multiple, overlapping abstraction layers in your test automation and ensure proper synchronization. Rely on explicit waits instead of implicit ones.
2. Don't treat AI and machine learning as black boxes. Learn how they work and always ask yourself, "How would I test this?"
3. Implement the Page Object Model (POM) to make tests more readable and maintainable. Also, understand how your business creates value.
4. Always get proper training before using any test automation tool. Make sure it's from a trusted source to avoid bad practices.
5. Collaborate with your whole team on testing challenges. Bring problems to everyone and brainstorm ways to solve them together.

10 actionable test automation tips from the experts featured in the "10 Quick Actionable Test"

6. Prioritize testability. If your application isn't testable, automation efforts will likely fail or be inefficient.
7. Integrate your tests into a CI pipeline and run them frequently. Design tests with CI in mind from the beginning.
8. Follow the Test Pyramid principle: write many unit tests, fewer integration tests, and only a small number of end-to-end UI tests.
9. Understand that BDD is about shared understanding and collaboration, not just automation tools. Engage both developers and business stakeholders.
10. There's no one-size-fits-all automation tool. Choose tools based on your product, goals, and team's needs.

More Advanced Topics

- [BiDirectional functionality | Selenium](#)
- [Event-driven Testing with Selenium BiDi | BrowserStack Docs](#)
- [Selenium WebDriver BiDi: Kismet Child of WebDriver Classic and Chrome DevTools Protocol | by Lana Begunova | Women in Technology | Medium](#)
- [15 Best Test Automation Trends for 2025 | LambdaTest](#)
- [Advanced Topics Archives - AI-Powered End-to-End Testing | Applitools](#)
- [Ecosystem | Selenium](#)
- [How to use Docker for Automation Testing? | Engati](#)
- [How to run Selenium Tests in Docker | BrowserStack](#)
- [Docker for QA Engineers. By Charithra Janani — Associate... | by Zone24x7 | Medium](#)

Important Resources

- [Upgrade to Selenium 4 | Selenium](#)
- [How To Upgrade to Selenium 4](#)
- [BiDirectional functionality | Selenium](#)
- [Selenium Archives - AI-Powered End-to-End Testing | Applitools](#)
- [Design Patterns](#)
- [Grid | Selenium](#)
- [The Best Test Automation Framework ismp4 on Vimeo](#)
- [Cypress, Playwright, Selenium, or WebDriverIO? Let the Engineers Speak!](#)
- [Intro to HTML and CSS | Udacity](#)
- [\(272\) Automation talks - YouTube](#)

Important Resources

- [Patterns of a “good” Test Automation Framework, Locators & Data! by Anand Bagmar](#)
- [\[Webinar Recording\] Design Patterns in Test Automation](#)
- [\[LightningTalks2020\] - Does Your Functional Automation Really Add Value? by Anand Bagmar](#)
- [To Automate or Not to Automate](#)
- [How to Create a Test Automation Framework from Scratch - Angie Jones](#)
- [Blog - Angie Jones](#)
- [Test Automation Career Path - Angie Jones](#)
- [autoamtoin frameworks - Google Docs](#)
- [automation anti-patterns - Google Docs](#)
- [Which tests should be automated - Google Docs](#)
- [Object oriented programming in software testing - Google Docs](#)

Important Resources

- [Fluent Interface Design Pattern in Automation Testing | LambdaTest](#)
- [All about TestNG Listeners in Selenium | BrowserStack](#)
- [Allure Listener](#)
- [Build Selenium Data-Driven Framework: Step-by-Step Guide | LambdaTestData Driven](#)
- [Testing Using JSON | Independent Software QA Testing Services](#)
- [Running a Single Test or Method With Maven | Baeldung](#)
- [Testing useful resources - Google Drive](#)
- [get ready for the job - Google Drive](#)
- [Generative AI: the future of testing? - Test Smarter, Not Harder Event](#)
- [Recordings and Slides - SeleniumConf & AppiumConf](#)

Important Resources

- [Webinar - How to use AI in Software Testing \[EN\] – YouTube](#)
- [Page Objects: You're Doing It Wrong - Titus Fortner | Senior Solution Architect at Sauce Labs – YouTube](#)
- [Optimizing Page Factory](#)